

RAPPORT DE PROJET

Aux Armes

Simon Beasse | Naïm Courbois

Mars-Mai 2026

Remerciement

Nous remercions le laboratoire LERIA de l'université d'Angers de nous avoir permis de réaliser ce projet dans des conditions optimales.

Un grand merci à notre référent M.Igor STÉPHAN, dont la connaissance technique et la bienveillance nous a permis de mener à bien le développement de ce projet.

Nous saluons également l'ensemble du corps enseignant de l'université d'Angers pour leur soutien théorique et pratique au cours de ces trois années de licence.

Sommaire

Introduction	4
I. Organisation du projet	
1. Initialisation du projet	5
2. Répartition du projet	6
3. Diagramme de Gantt	6
II. Initialisation du Projet et Paradigme du Design Piloté par les Données	
1. Genèse du projet et objectifs d'universalité	7
2. Architecture et modélisation polymorphe du plateau hexagonal	7
3. Algorithmique de génération du monde et contraintes du conteneur	8
III. Gestion et Comportements des unités	
1. Orientation	10
2. Unité/Comportement	13
3. Combat	15
IV. Modélisation de l'Économie : Villes, Bâtiments et Logique d'Urbanisme	
1. Implémentation du patron Prototype pour les extensions de contenu	18
2. Gestion sécurisée de la mémoire et encapsulation de la classe City	18
3. Intégration de l'Arbitre, de l'Économie et du Système de Factions	19
V. Refactorisation Majeure : Architecture MVC, Standard JSON et Patron Command	
1. Migration vers le standard JSON et optimisation du parsing	20
2. Refactorisation du Système de Requêtes : Patron Command et std::variant	20
3. Déploiement de l'Architecture MVC : Joueur, Arbitre et Moteur	21
VI. Interfaces	
1. L'Interface Graphique et l'Infrastructure Réseau Multijoueur	26
2. Interface Console	29
VII. Implémentation	
1. La construction de l'univers (Worldbuilding)	30
2. Direction Artistique et Asset Pipeline	30
3. Équilibrage des données (Game Design)	30
VIII. Test	
1. La nécessité d'une validation rigoureuse	32
2. Isolation et Mocking des données	33
3. Couverture des Tests	33
IX. Intelligence Artificielle par Apprentissage Profond par Renforcement	
1. Objectifs de l'expérimentation et framework technologique	33
2. Architecture de liaison (Bindings C++ / Python)	33
3. Difficultés techniques rencontrées et optimisations systèmes	34
4. Démarche d'apprentissage et bilan pédagogique	35
Conclusion	36
Webographie.....	37
Annexes.....	38

Introduction :

Pour conclure notre formation en Licence 3 d'Informatique à Angers, nous avons pu effectuer un stage de huit semaines au sein du laboratoire LERIA de l'Université. Sous la supervision de M. Igor STEPHAN, nous avons entrepris la mise en place d'un moteur de jeu de guerre universel, dont voici l'intitulé :

“Ce sujet porte sur la modélisation et la réalisation d'un « universal wargame » en tour par tour sur un terrain hexagonal. L'objectif de ce stage est double : premièrement, définir un ensemble de primitives simples pour pouvoir modéliser la plus grande part des wargames (mouvements, ravitaillements, combats, etc.) ; deuxièmement, concevoir un module tactique. Ce sujet pourra comporter une partie interface graphique ou OpenGL pour le rendu du champ de bataille. L'application sera réalisée en C/C++.”

Notre souhait pour ce stage étant l'approfondissement de nos connaissances en C++, ce sujet nous a permis d'appliquer les différentes connaissances acquises au cours des trois années de licence et de découvrir d'autres facettes de ce langage. De plus, notre attrait pour l'univers vidéo ludique, et plus particulièrement les jeux tels que civilisation ont grandement orienté notre approche vis-à-vis de ce projet.

L'objectif était donc de modéliser un universal wargame en C++ sur un terrain hexagonal. Nous avons choisi ici d'orienter notre projet vers un mélange entre Civilisation, impliquant un système de gestion de ressource et Total War, pour l'aspect tactique du wargame, permettant d'avoir un jeu accessible tout en gardant les mécaniques qui ont fait le succès des wargames. Le plus important étant de rendre ce moteur générique, afin de pouvoir instancier différentes époques ou différents genres sur celui-ci.

La durée du projet étant de huit semaines, il est compliqué de rendre un jeu parfaitement fonctionnel, nous nous sommes donc orientés vers la mise en place d'une architecture modulaire qui pourra simplement être améliorée. De plus, notre approche a été de se focaliser sur le jeu en lui-même et non sur l'interface graphique, n'ayant pas suivi le cours de Synthèse d'image.

Au cours de ce rapport, nous allons vous présenter notre méthode de travail, notre architecture qui a évolué au fil du temps, ainsi que les difficultés rencontrées. Et pour finir nous vous présenterons les améliorations qui pourraient être apportées pour que ce projet devienne un jeu complet.

I. Organisation du projet :

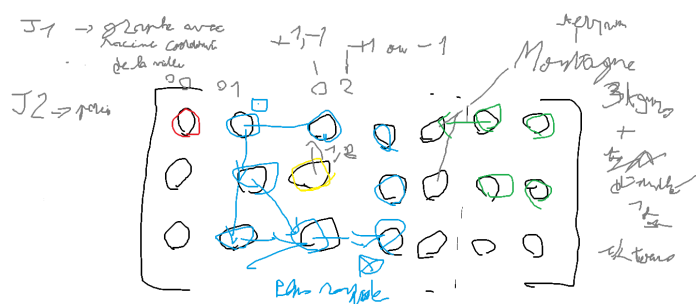
1.1. Initialisation du projet

Avant de se lancer tête baissée dans le projet, nous avons d'abord, durant une semaine, posé à plat tous les défis de ce projet.

Le commencement fut de définir notre choix de jeu et l'architecture, en définissant les indispensables à ajouter et ce qui pouvait être mis de côté sans perdre l'essence du wargame. Nous sommes arrivés à cette conclusion :

- Le jeu possède un plateau de tuiles avec les joueurs.
- Les joueurs quant à eux auront des unités et des villes, ou plus globalement un inventaire d'objets.
- Nous aurons également besoin d'une interface graphique, permettant d'avoir une vue plus concrète du jeu.
- Et enfin, l'objectif était de se concentrer sur le jeu en lui-même, nous prévoyons une potentielle mise en place d'une IA si le temps nous le permet.

Pour ce faire, nous avons créé des croquis pour permettre de mieux se coordonner et de partir vers la même base architecturale, vous pouvez voir ci-dessous l'un des premiers croquis.



Cette phase terminée, nous avons essayé de définir le système de jeu et de décrire toutes les mécaniques de base telles que les différentes phases du jeu, les conditions de victoire, les systèmes de factions, d'expérience, de terrain et d'autre que vous découvrirez au cours de ce rapport.

Enfin pour finaliser cette préparation, nous nous sommes renseignés sur les mécaniques caractéristiques d'un wargame afin de vérifier si nous n'avons rien oublié. A la suite de nos recherches nous avons bien un système de morale, de surprise et de chaîne d'approvisionnement à implémenter. Pour nous démarquer d'un wargame classique nous avons également prit le parti d'hériter de divers systèmes issus de la licence civilisation, avec tel que la gestion de ressource et la gestion de ville et de bâtiments évolutive.

Le cadre posé nous nous sommes lancés dans une première approche nous conduisant à un code trop contextuel, en basant nos différentes classes sur l'implémentation que nous voulions faire, c'est-à-dire un wargame spatial, nous avons oublié l'objectif principal,

l'universal wargame. Cependant, cette approche limitait le code à une simple application de notre space-wargame.

Avec l'appui de M.STEPHAN, nous avons repensé l'abstraction pour essayer d'être plus universel, cette nouvelle approche nous a permis d'améliorer l'architecture globale et de pouvoir créer un universal wargame.

1.2. Répartition du projet

Afin de pouvoir optimiser notre temps, il fût impératif de scinder le projet en différentes tâches et de se les attribuer de façon à pouvoir développer, indépendamment de l'autre, du code.

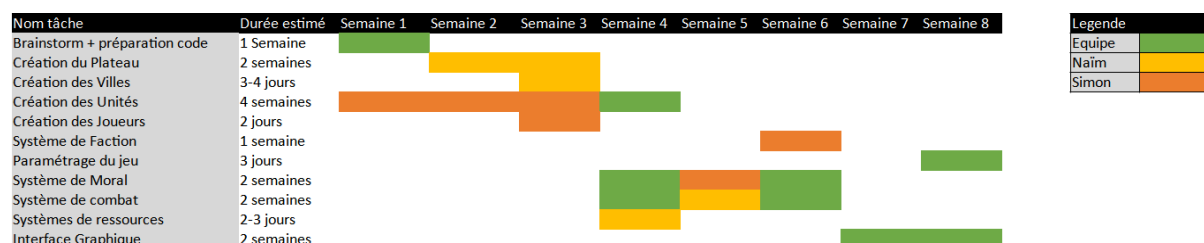
Nous sommes arrivés à cette conclusion :

- L'étudiant, Simon BEASSE effectuera le développement de la partie unité, ce qui implique la mise en place du système de comportement ainsi que le développement du combat et de ces mécaniques.
- L'étudiant, Naïm COURBOIS quant à lui développera le plateau, ce qui implique la mise en place des tuiles, des villes et des bâtiments ainsi que le système de ressources.
- Dans un dernier temps, nous mettrons en place le moteur du jeu, impliquant la mise en place des joueurs et de l'arbitre.
- Et pour finir Naïm COURBOIS s'occupera de la partie interface graphique et Simon BEASSE de la partie interface console.

Pour ne pas créer de conflit entre nos différents codes, nous avons utilisé le gestionnaire de version git et l'hébergeur github pour pouvoir créer différentes branches et travailler chacun sur le même code, tout en pouvant récupérer les avancer l'un de l'autre. Nous apportant l'avantage majeur de pouvoir fusionner nos codes de manière propre en regardant ce que chacun avait produit pour garder le meilleur code pour notre jeu.

1.3 Diagramme de Gantt

Vous pouvez voir ici les objectifs que nous nous étions fixés pour le projet, via ce Diagramme de Gantt.



En conclusion nous reviendrons sur celui-ci pour comparer notre estimation et la réalité.

II. Initialisation du projet et choix paradigme Design Piloté par les Données (Data-Driven)

2.1 Début du projet et objectifs

La première phase de notre travail a été de poser les bases du projet en développant le fichier principal main, ainsi que les fichiers de base `jeu.hh` et `jeu.cc` qui posséderont la structure globale du plateau de jeu. L'objectif est de construire un universal wargame et de pouvoir créer n'importe quelle instance de jeu à partir de notre code pour créer plusieurs univers. Le défi d'architecture qui est posé est de ne pas avoir de donnée de gameplay écrite en dur dans le code.

Pour pouvoir faire cela, le choix s'est porté vers une programmation pilotée par les données ou Data-Driven Design. Ce paradigme a pour objectif d'avoir toutes les configurations, les règles du jeu et tout ce qui touche au jeu dans des fichiers externes au code source. Cela permet deux avantages :

- Pouvoir modifier l'équilibrage du jeu et créer plusieurs instances dans différents univers sans avoir besoin de recompiler à chaque fois.
- Utiliser la Fabrique comme patron de conception (Factory Design Pattern) qui lit les fichiers externes pour instancier les bons objets lors de l'exécution du code. Le moteur devient donc universel et permet de charger un univers dans l'espace ou de pirates via le même jeu.

2.2. Architecture du plateau en hexagone

Pour tester cela, nous avons créé un premier fichier de configuration au format texte nommé `config_espace` pour pouvoir tester la création des différentes tuiles. La structure de lecture du fichier est stricte : nom de la tuile, symbole d'affichage, trois booléens pour définir si elle est franchissable par les trois types d'unités universelles (terrestre, maritime, aérienne), ainsi que sa probabilité d'apparition sur le terrain (la somme doit atteindre 100). A côté de cela nous avons implémenté une classe `Unite` basique dans le fichier jeu qui a pour objectif de tester les interactions sur le plateau.

Le plateau est créé selon une hiérarchie précise :

- `Hexa` (classe mère abstraite) : Elle définit l'interface universelle de n'importe quelle tuile du jeu en les caractérisant par un type spécifique, un coût de déplacement, un symbole d'affichage ainsi que les infos de franchissement par les unités.
- `Tuile_Configurable` (classe fille concrète) : Elle hérite de la classe `Hexa` et représente l'objet instancié dont les attributs viennent du fichier de configuration.

Optimisation de la RAM :

Dû à la grande quantité de données contenus dans le jeu et par les tuiles, nous avons ajouté une structure pour optimiser la mémoire vive utilisée. Au lieu de dupliquer les attributs redondants de base des tuiles, nous avons enregistré ses données dans la structure nommée `tuile_data`. Chaque objet `Tuile_Configurable` reçoit un pointeur vers l'instance unique de `tuile_data` qui lui correspond. Les données sont donc partagées et cela évite d'avoir des copies identiques en RAM.

Actuellement dans le projet, nous utilisons uniquement `Tuile_Configurable` qui sont les données issues des fichiers, mais grâce à l'interface `Hexa`, on peut tout à fait créer des tuiles qui ne sont pas configurées par fichier, notamment les bordures de carte ou autres zones qui sont communes pour toutes les instances du jeu. C'est pour cela que la matrice de jeu utilise uniquement des objets `Hexa`, pour permettre une évolutivité du projet.

2.3. Génération du monde et contraintes

La génération procédurale du plateau se fait grâce à la classe abstraite `WorldGenerator` qui définit l'interface de création des mondes. Cela permet d'avoir une implémentation sur de l'aléatoire ou sur des générateurs de bruit. Suite aux suggestions de notre encadrant M.STEPHAN, nous avons ajouté cette approche procédurale en utilisant un algorithme basé sur le bruit de Perlin (cf. annexe A.2). Le choix a été fait d'utiliser la library open-source `FastNoiseLite` pour sa simplicité d'utilisation et son intégration rapide, car elle est dans un unique fichier .hh récupéré depuis github. Pour utiliser cette librairie de manière Data-Driven, nous avons ajouté dans les fichiers de configuration, des presets de mondes (ex: preset Perlin ou Random).

La classe `WorldFactory` hérite de `WorldGenerator` et permet d'utiliser la méthode `chargerConfig` pour lire le fichier, créer le catalogue des tuiles et générer les cases sur le terrain. Elle lit les paramètres externes tels que les seuils de chaque tuile et configure la génération avant de créer la carte.

La classe `Board` possède le plateau en prenant en paramètre ses dimensions, ainsi que l'instance de `WorldGenerator`, pour l'initialisation et l'affichage. Le stockage utilise une matrice de vecteurs :

```
std::vector<std::vector<std::unique_ptr<Hexa>>> matrix;
```

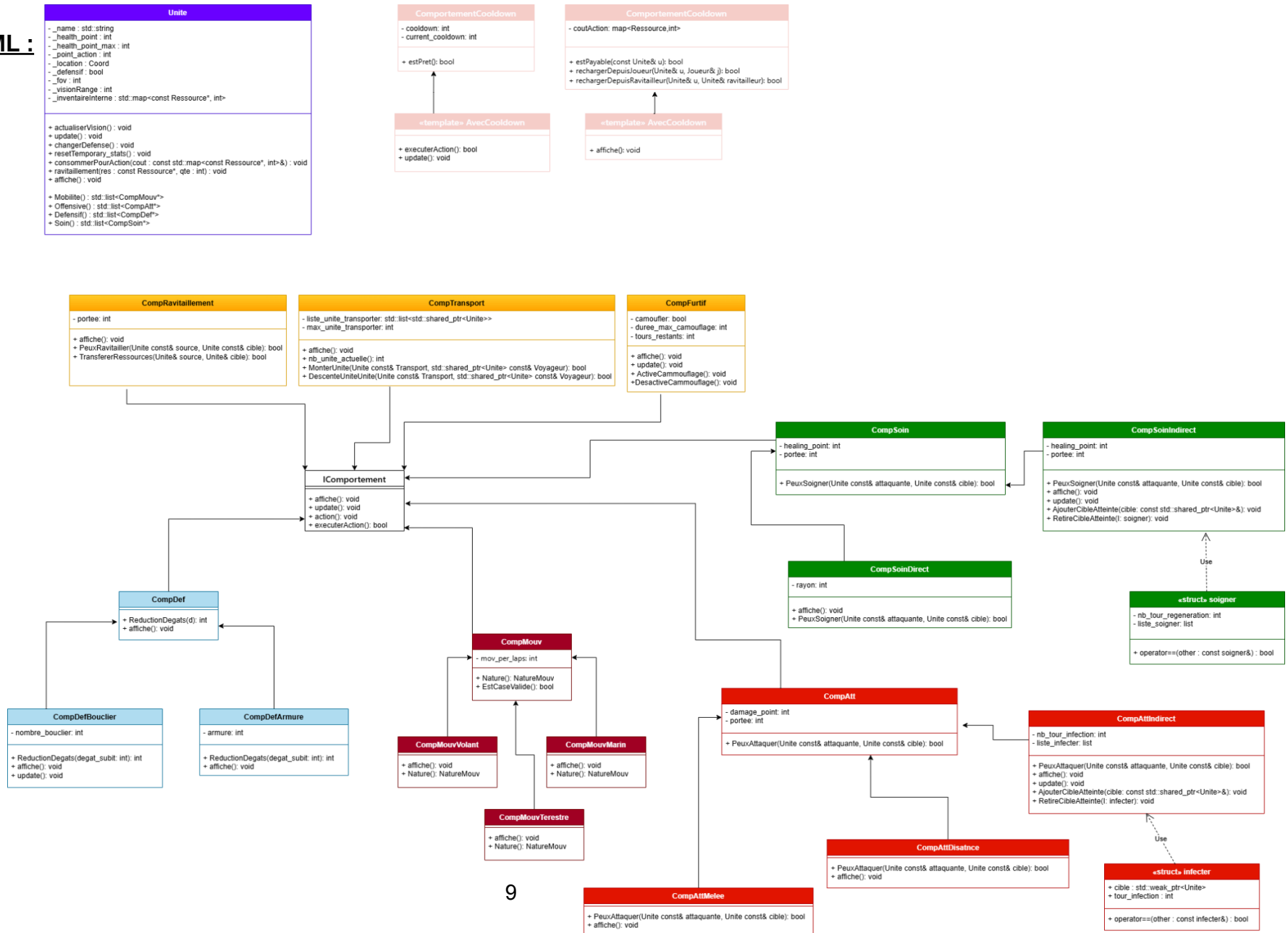
L'utilisation de cette structure permet d'obtenir un accès direct aux cellules, en temps constant, donc avec comme complexité $O(1)$.

Cela utilise la méthode `getCell(i,j)` qui permet d'optimiser les performances. L'utilisation des pointeurs uniques avec `std::unique_ptr` est ici obligatoire. `Hexa` étant une classe abstraite, elle ne peut pas être instanciée directement, `unique_ptr` permet donc de stocker de façon sécurisée l'adresse mémoire des `Hexa`, ici `Tuile_Configurable`.

Le fichier main instancie la `FileFactory`, charge le fichier de configuration puis crée l'objet `Board`. Le projet est compilé via un `MakeFile` avec le compilateur g++. Nous avons également ajouté des méthodes pour la génération avec `postGeneration` qui vérifie après la première génération que le plateau respecte les prérequis donné par la configuration, tel que la présence d'un nombre minimum de tuiles spécifiques.

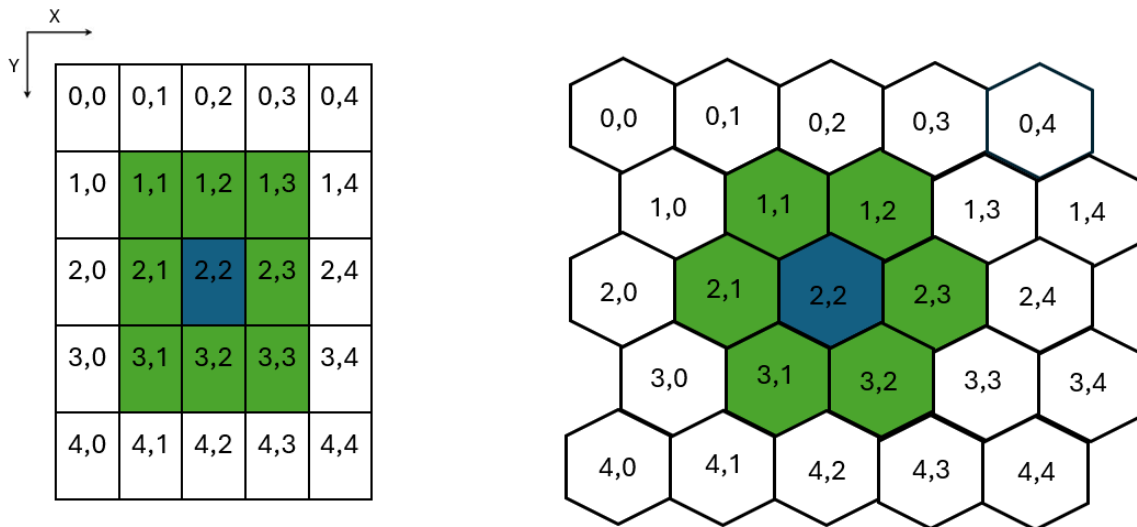
III. Gestion et Comportements des unités :

Diagramme UML :



Avant de vous présenter en détail les unités, nous allons rappeler une petite précision. Un wargame se déroule sur un plateau composé d'hexagone, nous avons dû mettre en place des stratégies pour retranscrire ceci en code en partant d'un tableau de case représenter sous forme de liste de liste.

Le premier problème, on peut considérer que chaque case d'une liste de liste à huit voisins mais un hexagone ne peut avoir que six voisins. Pour les exemples qui suivront nous prendrons toujours les coordonnées 2,2 comme référence. Ci-dessous vous pouvez voir la différence entre les voisins d'une case sur un plateau rectangulaire et sur un plateau hexagonal.



Pour résoudre ce problème nous avons créé un fichier **orientation** qui aura pour but de gérer les interactions entre coordonnées. Nous allons maintenant vous expliquer plus en détail son fonctionnement et la manière dont nous avons géré ce problème.

3.1.Orientation :

Dans un wargame le positionnement peut énormément jouer sur le résultat des combats, et les interactions entre les divers éléments du jeu. Pour gérer ces interactions nous avons mis en place diverses fonctions représentant le cœur de notre système de repérage sur la carte. Le module **orientation** joue un rôle central : il fait le lien entre notre grille stockée en liste de listes et la logique géométrique propre aux plateaux hexagonaux. Nous l'avons découpé en trois grandes parties.

3.1.1. Représenter l'espace et calculer les voisins

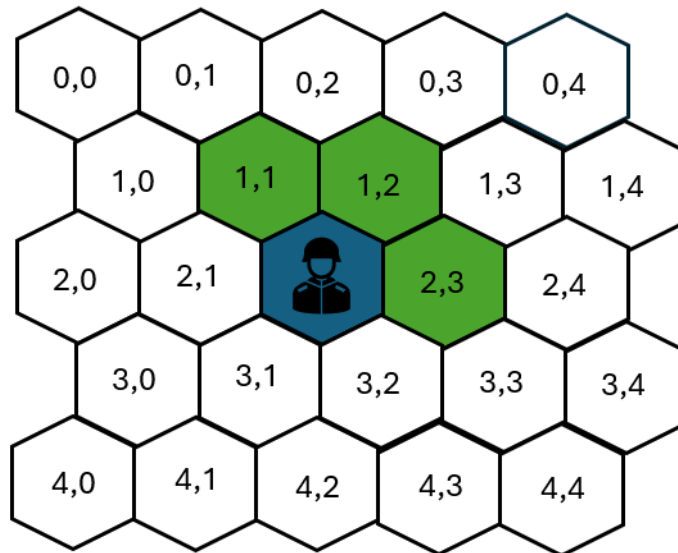
Pour les positions, nous avons défini un alias de type **Coord** qui est simplement une paire d'entiers (`std::pair<int, int>`). Les directions sont représentées par une énumération **direction** avec les six orientations possibles d'un hexagone (Nord-Ouest, Nord-Est, Ouest, Est, Sud-Ouest, Sud-Est).

La vraie difficulté a été de calculer les cases adjacentes. Sur une grille hexagonale, les lignes sont décalées une ligne sur deux, donc on ne peut pas juste ajouter +/- 1 à chaque coordonnée. La fonction **Voisins** gère cela en vérifiant la parité de la coordonnée pour ajuster les décalages selon si on est sur une ligne paire ou impaire. À partir de là, nous

avons codé `case_adjacentes`, qui permet de récupérer toutes les cases dans un certain rayon autour d'un point, utile pour beaucoup de mécaniques de jeu.

3.1.2. Le champ de vision

Au départ dans une logique d'efficacité la fonction `Case_visible`, prenait une direction et renvoyait les trois cases face à elle.



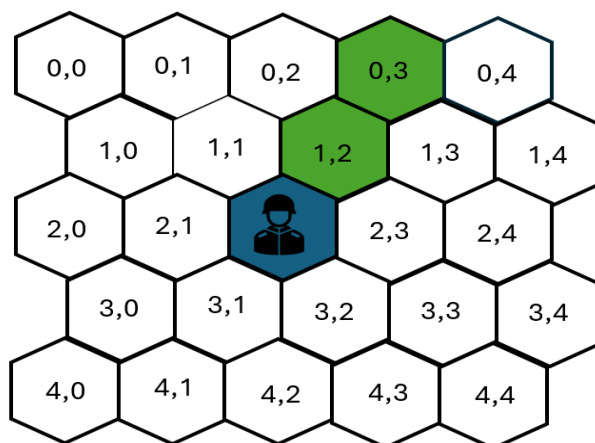
Légende :

L'unité  est en position 2,2.

direction = direction :: nord_est

Seulement dans un wargame, ce que voit une unité change tout. La vision est un élément fort du gameplay et doit différer selon les unités. Nous avons donc modélisé un cône de vision via deux fonctions :

- `Case_visible` qui a été remanié pour calculer les cases visibles à une distance de un selon la direction regardée et un angle de vue. Concrètement, elle recentre mathématiquement le cône pour ne garder que les bons voisins parmi les six possibles.
- `ConeVision` fut ajouté pour étendre cette logique sur plusieurs cases en propageant le champ de vision jusqu'à une portée donnée. Pour éviter de traiter plusieurs fois la même case et garder des performances correctes, nous avons aussi protégé le code des doublons pour que lors du parcours des cases que la fonction renvoi on fasse moins de calcul.



Légende :

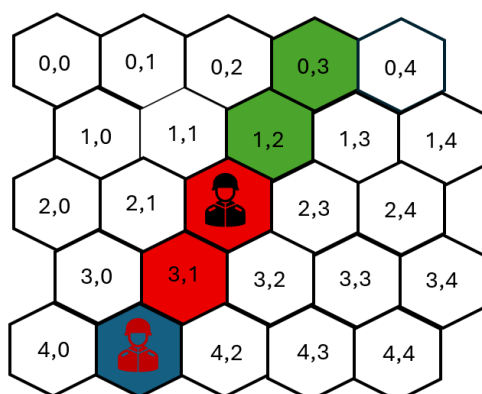
L'unité est en position 2,2.

direction = direction :: nord_est fov = 1 range = 2

3.1.3. Les mécaniques de combat

L'orientation intervient aussi directement dans les combats. La fonction `directionVers` permet à une unité de savoir automatiquement dans quelle direction elle doit se tourner pour cibler une case précise, pratique pour ne pas avoir à gérer ça à la main partout dans le code.

Ensuite, la fonction `avantage_attaque` implémente directement une mécanique de jeu. Elle met en place le principe d'attaque surprise : elle vérifie si l'attaquant se trouve en dehors du cône de vision du défenseur. Si c'est le cas, la fonction renvoie true, ce qui accorde un bonus lors du calcul des dégâts. C'est une petite mécanique simple mais qui ajoute vraiment de la profondeur tactique.



Légende :

L'unité est en position 2,2.

direction = direction :: nord_est

fov = 1 range = 2

L'unité est en position 4,1.

direction = direction :: nord_est

fov = 1 range = 2

L'unité rouge prendra par surprise l'unité noire car elle attaque depuis une case non visible par l'unité noire.

3.2.Unité/Comportement :

3.2.1. Introduction et Évolution de la Conception

Une fois la structure géométrique du plateau posée, il nous a fallu réfléchir à la manière de modéliser les entités qui évoluent dessus. L'ambition d'un moteur « universal wargame », capable de s'adapter à des règles, des époques et des types d'affrontements variés, a directement orienté nos choix d'architecture.

Au départ, nous sommes partis sur une approche par héritage classique. L'idée était d'avoir une classe de base *Unite* dont dérivent des classes spécialisées comme *Avion*, *Bateau*, *Tank* ou *Fantassin*. Assez vite, nous nous sommes heurtés aux limites de ce modèle. Comment gérer proprement un véhicule amphibie, à la fois terrestre et marin ? Ou une unité de soutien capable de soigner et de transporter simultanément, sans dupliquer du code ou multiplier les sous-classes à l'infini ?

Il est devenu évident que l'héritage simple n'était pas adapté à ce que nous voulions construire. C'est ce constat qui nous a amenés à basculer vers une architecture orientée composants, articulée autour du concept de Comportements.

Dans ce nouveau modèle, *Unite* devient un conteneur générique regroupant des attributs universels (points de vie, points d'action, orientation). Ses capacités spécifiques lui sont ensuite injectées dynamiquement via une liste de comportements polymorphes (mouvement, attaque, défense, soin, etc) selon les besoins du scénario.

Cette même logique de flexibilité s'est imposée pour la gestion des temps de recharge (cooldowns). Initialement, nous n'avions prévu cette mécanique que pour trois comportements bien précis : le soin indirect, l'attaque indirecte et le camouflage. Mais en y réfléchissant davantage, nous avons réalisé qu'il n'y avait aucune raison de la limiter à ces deux cas. Une attaque directe ou une compétence de déplacement spéciale pourraient tout autant nécessiter un temps de récupération. Pour généraliser cette fonctionnalité sans alourdir la hiérarchie existante, nous avons eu recours aux templates C++. La classe *AvecCooldown* suit une logique proche du patron de conception Décorateur : elle vient s'envelopper autour de n'importe quel comportement existant pour lui ajouter une gestion de récupération, de façon totalement transparente.

Enfin, un wargame se doit d'intégrer une dimension logistique. La chaîne d'approvisionnement est un élément stratégique central dans toute simulation militaire sérieuse, et nous souhaitons que notre moteur permette de modéliser ce type de contrainte. Nous avons donc couplé notre système de comportements à un mécanisme de consommables, via le template *AvecConsommable*. Celui-ci permet d'associer un coût en ressources à n'importe quelle action. En complément, le comportement *CompRavitaillement* introduit la possibilité de créer des unités logistiques, capables de transférer munitions ou carburant aux troupes. Le moteur offre ainsi la possibilité de concevoir des scénarios où la rupture des lignes d'approvisionnement peut à elle seule faire basculer l'issue d'une bataille.

3.2.2. Les familles de comportements de base

Pour donner vie à cette architecture, nous avons organisé l'ensemble des compétences autour d'une interface commune, *IComportement*, qui expose une méthode virtuelle pure *affiche()* pour le débogage et l'interface console, ainsi que des points d'entrée

comme `update()` et `action()`. À partir de là, nous avons défini quatre grandes familles de comportements, ainsi que quelques modules spéciaux :

- **Le Mouvement (`CompMouv`)** gère la mobilité des unités sur la carte. Nous l'avons décliné en trois classes selon le milieu d'évolution : `CompMouvVolant`, `CompMouvMarin` et `CompMouvTerrestre`. La méthode `EstCaseValide` s'appuie sur la fonction `case_adjacentes` pour vérifier que la tuile ciblée se trouve bien dans le rayon de déplacement autorisé.
- **L'Attaque (`CompAtt`)** regroupe les capacités offensives. On distingue le corps-à-corps `CompAttMelee` avec une portée fixe de 1, l'attaque à distance `CompAttDistance` avec une portée minimale et maximale, et l'attaque indirecte `CompAttIndirect`. Cette dernière fonctionne sur un principe de dégâts dans le temps : les cibles touchées continuent de subir des dégâts à chaque tour, gérés via une liste de structures `infecter` mise à jour à chaque appel d'`update()`.
- **La Défense (`CompDef`)** encapsule les mécanismes de réduction des dégâts reçus. Deux approches coexistent : `CompDefArmure`, qui applique une réduction fixe sur chaque attaque, et `CompDefBouclier`, qui absorbe intégralement une attaque en consommant un point de bouclier, lequel se régénère automatiquement d'un point par tour.
- **Le Soin (`CompSoin`)** permet de restaurer les points de vie des alliés. Comme pour l'attaque, il existe une version directe `CompSoinDirect`, à effet instantané sur un rayon donné et une version indirecte `CompSoinIndirect`, qui applique une régénération progressive sur plusieurs tours via une structure interne `soigner`.
- **Les comportements spéciaux** viennent enrichir les possibilités tactiques. `CompTransport` permet d'embarquer et de débarquer des unités alliées compatibles à proximité. `CompFurtif` active un état de camouflage masquant l'unité pendant une durée définie. Enfin, `CompRavitaillement` matérialise la logistique en permettant le transfert de ressources depuis une unité de soutien vers une unité de combat à portée.

3.2.3. L'extensibilité par la programmation générique

En développant ces mécaniques, nous avons rapidement voulu éviter de surcharger la hiérarchie d'héritage d'`IComportement` avec des variantes combinatoires, créer une classe `CompAttMeleeAvecCooldown`, puis une autre `CompAttMeleeAvecConsommable`, etc., aurait vite rendu le code ingérable. Nous avons donc eu recours aux templates C++ pour gérer ces fonctionnalités de manière transversale.

Nous avons conçu deux structures templates : `AvecCooldown` et `AvecConsommable`. Elles fonctionnent comme des décorateurs génériques résolus à la compilation. En utilisant les variadic templates (`template <typename... Args>`) et le perfect forwarding (`std::forward`), ces classes peuvent s'envelopper autour de n'importe quel comportement héritant d'`IComportement`, tout en acceptant ses arguments de construction d'origine de façon transparente.

Concrètement, ajouter une contrainte de recharge sur un comportement existant ne demande aucune modification de ce comportement : `executerAction()` vérifie la disponibilité via `estPret()`, déclenche l'action, puis réinitialise le compteur. Le template

`AvecConsommable` suit la même logique, en vérifiant cette fois que l'unité dispose des ressources nécessaires avant d'autoriser l'exécution.

3.2.4. L'instanciation des unités par fichier de configuration (Data-Driven et Factory)

L'un de nos objectifs dès le départ était de pouvoir ajouter ou modifier des unités sans toucher au code C++ ni recompiler. Pour y parvenir, nous avons mis en place une architecture pilotée par les données (*Data-Driven*), en combinant des fichiers de configuration JSON avec le patron de conception Fabrique (*Factory*).

Deux composants principaux assurent ce fonctionnement dans le module unite :

- `JsonUniteReader` s'occupe de parser le fichier JSON. Elle en extrait les statistiques de base (points de vie, points d'action, vision, champ de vision, poids, symboles) et instancie dynamiquement les comportements associés via la fonction `createComp`. C'est à ce niveau que la fabrique décide si un comportement doit être enveloppé par `AvecCooldown` ou `AvecConsommable`, simplement en détectant la présence des clés correspondantes dans le JSON. Elle gère également l'attribution des grades `Rank_Commandant` avec ses bonus passifs, ou `Rank_Regulier`.
- `UniteFactory` centralise le catalogue des unités chargées sous forme de pointeurs partagés (`std::shared_ptr<Unite>`). Quand une unité doit être déployée en cours de partie, la méthode `create(std::string type)` recherche le modèle correspondant dans ce catalogue et en retourne une copie via le pattern Prototype, implémenté dans `Unite::clone()`.

Cette séparation stricte entre les données et la logique du moteur offre une extensibilité très appréciable en pratique. Un concepteur peut définir un Archer ou un Chasseur Furtif en éditant simplement un fichier JSON, sans jamais ouvrir le code source, ce qui illustre bien la philosophie modulaire que nous avons cherché à tenir tout au long du projet.

3.3. Combat :

3.3.1. Philosophie du Système de Combat

Après avoir posé les bases de la carte et de l'architecture des unités, il nous fallait concevoir le cœur de l'expérience de jeu : le système de combat. Dans un wargame, la façon dont les affrontements sont résolus conditionne à la fois l'équilibre tactique et le réalisme de la simulation.

Au niveau de son interface de base, notre moteur expose deux actions via la classe statique `Combat` : Le soin (`Combat::heal`) et L'attaque (`Combat::fight`)

Pour aller au-delà de cette base et apporter une vraie profondeur stratégique, nous avons construit deux sous-systèmes complémentaires : la chaîne de commandement et la gestion du moral. Le premier traduit l'influence des officiers sur le terrain. Les commandants transmettent des auras tactiques, offensives, défensives ou médicales, aux unités sous leurs ordres, simulant la cohésion d'une force armée organisée. Le second représente, selon nous, l'apport le plus intéressant de notre moteur en termes de réalisme : contrairement aux wargames classiques où une unité combat avec la même efficacité jusqu'à son dernier point

de vie, notre système évalue en permanence l'état psychologique de chaque entité. Ses performances dépendent directement de ses succès et revers récents au combat.

3.3.2. Mécanique de Résolution et Impact de la Topologie

Lorsqu'une attaque est déclenchée via `Combat::fight`, le moteur commence par une phase de validation : il vérifie que l'attaquant n'est pas en posture purement défensive, que le type d'attaque passé en paramètre par le joueur fait partie des comportements de l'attaquant, et que la cible n'est pas camouflée par un composant de furtivité.

Une fois ces vérifications passées, le calcul des dégâts entre en jeu et c'est là que le lien avec le module orientation devient concret. La fonction `avantage_attaque` est appelée pour déterminer si l'attaquant se trouve en dehors du champ de vision du défenseur, ou si l'attaque provient d'une unité camouflée. Si c'est le cas, l'action est considérée comme une attaque de flanc ou par surprise. Les dégâts de base sont alors multipliés par 1.5 et l'impact sur le moral du défenseur est fortement amplifié. Dans un affrontement frontal classique, les dégâts restent inchangés et l'effet sur le moral demeure modéré.

Avant d'appliquer les dégâts finaux, le moteur parcourt la liste des comportements défensifs du défenseur (`CompDef`). Chaque module vient ajuster les dégâts nets que l'entité encaisse réellement.

3.3.3. Modélisation du Moral

Le moral de chaque unité évolue sur une échelle bornée entre -20.0 et +20.0, modifiable simplement dans le code en changeant la valeur des constantes `MIN_MORAL` et `MAX_MORAL`. À chaque combat, la méthode `EffetMoral` calcule un ratio entre 0.0 et 1.0 reflétant l'état psychologique de l'unité :

$$\text{ratio} = (\text{moral} - \text{MIN_MORAL}) / (\text{MAX_MORAL} - \text{MIN_MORAL})$$

Ce ratio détermine ensuite dans quel palier psychologique se trouve l'unité, chacun appliquant un coefficient multiplicateur sur ses dégâts via `setTemporary_damage` :

- **Fuite** (`ratio < 0.14`) : Effondrement, l'unité déserte immédiatement le champ de bataille, ses points de vie sont annulés.
- **Panique** (`ratio < 0.28`) : Malus offensif majeur (coefficient 0.5) et perte immédiate de 20% de santé.
- **Peur** (`ratio < 0.42`) : Stress réduisant les capacités offensives (coefficient 0.8).
- **Courage** (`ratio > 0.58`) : Bonus d'assurance standard (coefficient 1.2).
- **Héroïsme** (`ratio > 0.72`) : Moral excellent, bonus offensif massif (coefficient 1.5).
- **Fatigue** (`ratio > 0.86`) : À l'inverse, un excès d'activité applique un léger malus d'épuisement (coefficient 0.9).

Le moral est également réactif : si une unité parvient à bloquer la totalité des dégâts entrants (dégâts nets ≤ 0), elle regagne de l'assurance (`AugmentationMoral`), tandis que son attaquant voit le sien chuter face à l'échec de son assaut (`DiminutionMoral`).

3.3.4. La Chaîne de Commandement

Pour simuler une structure militaire réaliste, nous avons mis en place un système d'auras de commandement dans la fonction `BuffCommandant`. Avant de résoudre un affrontement, le moteur vérifie via un transtypage dynamique si l'unité concernée est une unité régulière liée à un commandant actif sur le terrain. Si c'est le cas, il extrait le grade de cet officier sous forme de `Rank_Commandant` et parcourt sa liste de bonus passifs pour les appliquer en temps réel :

- `BonusDegat` : Augmente temporairement la puissance d'attaque de l'unité subordonnée pour le calcul en cours.
- `BonusDefense` : Injecte des points de vie temporaires proportionnels à la santé maximale de l'entité, pour absorber les attaques entrantes.
- `BonusVie` : Active un indicateur permettant, si l'unité survit au combat, de déclencher `SoinDuCommandant` et de restaurer une partie des points de vie perdus.

Cette mécanique illustre bien la puissance du polymorphisme en C++ : la classe `Combat` manipule uniquement des interfaces abstraites (`IBonus`) et délègue le calcul précis aux classes concrètes. On obtient ainsi des lignes de front où la présence d'un commandant peut réellement faire basculer l'issue d'un engagement.

IV. Modélisation de l'économie : Villes et Bâtiments

4.1. Implémentation du patron Prototype

Nous avons ensuite étendu le moteur en y ajoutant la gestion de ressource (ex: poussiereDEtoile PE défini par un nom et un symbole) et des bâtiments (ex: ExtracteurDEtoile Or dix poussiereDEtoile dix définis par son nom, symbole, dictionnaire de ses coûts et de sa production). La classe **Batiment** est définie par un nom et un dictionnaire via `std::map` associant la référence de la ressource à un entier pour spécifier son coût.

Sachant que l'on doit créer un wargame universel, le moteur de jeu ne peut pas connaître lors de la compilation les bâtiments qui seront créés par les joueurs. Pour résoudre ce problème d'instanciation, nous avons implémenté de façon combinée les patrons de conception Prototype (l'utilisation de `clone` au lieu de `new`) et Fabrique (catalogue des Prototypes).

```
// Base abstraite qui impose la méthode virtuelle clone
class Batiment {
protected:
    std::string nom;
    std::string symbole;
    std::map<std::string, int> couts;
public:
    virtual ~Batiment() = default;
    virtual std::unique_ptr<Batiment> clone() const = 0; // Méthode Prototype
};
```

La classe **Batiment** est ici une classe mère abstraite qui impose la méthode virtuelle `clone()` et retourne un pointeur unique `std::unique_ptr<Batiment>`. Cela permet de créer de nouveaux objets en copiant le modèle de référence initialisé en mémoire, permettant d'éviter le mot-clé `new` sur une classe concrète.

La classe **BatimentRessource** hérite de **Batiment** et y ajoute des attributs spécifiques liés aux ressources extraites et la Factory possède le catalogue des bâtiments que l'on peut créer. Lors du lancement du jeu, **BatimentConfigReader** lit le fichier de configuration et charge les prototypes dans la fabrique. Lorsque l'on veut créer un **Batiment**, il est recherché dans le catalogue et appelle la méthode `clone()` sur le prototype correspondant.

Avec le polymorphisme, la fabrique fonctionne sans connaître la classe fille qu'elle va cloner. Cela permet une optimisation de la RAM où seuls les prototype de référence sont gardés de manière persistante et chaque joueur reçoit une copie physique de ses propres attributs.

4.2. Gestion sécurisée de la mémoire

L'utilisation de `std::make_unique` est ici privilégiée pour l'instanciation des pointeurs. Il permet de sécuriser le code en faisant l'allocation mémoire et la construction du pointeur en une seule étape. Cela élimine les risques de fuites mémoire et rend le code plus lisible.

Dans la classe `City`, la liste des bâtiments construits est sous la forme d'un vecteur de pointeurs intelligents pour garantir la gestion de la RAM en cas de destruction de la ville. Pour la manipulation de ceci, nous optons pour le type `size_t` qui garantit une gestion de taille d'objets et sécurise les boucles en interdisant les valeurs négatives. La classe `City` possède également des méthodes pour ajouter de nouveaux bâtiments et savoir si la ville est une capitale.

4.3. Ajout de l'arbitre, économie et système de factions

Le moteur possède donc la logique de construction des différentes structures en vérifiant les coordonnées (x,y) de la matrice pour savoir si la tuile ciblée est constructible et si le joueur possède les ressources requises. Les `Factory` sont initialisées dans le main en leur donnant les chemins de configuration qui rendent le wargame personnalisable. Chaque joueur possède un nom et un inventaire sous la forme d'une map, liant un objet `Ressource` à un entier. Des méthodes spécifiques permettent au joueur de contrôler ses ressources.

Toutes ses actions sont validées dans la classe `Arbitre` permettant d'avoir la logique de contrôle en dehors du joueur. Pour des actions comme `buildCity`, `peutAmeliorerVille` ou `estDansTerritoire`, l'arbitre suit des étapes précises :

- 1) Vérifier que les coordonnées sont valides et dans les limites du plateau
- 2) Récupère la tuile correspondante pour voir ses attributs et prérequis.
- 3) Vérifie que le joueur possède les prérequis en ressource ou point d'action.
- 4) Il retourne un booléen autorisant ou non l'action

La configuration du jeu se fait via la classe `GameConfig`, qui possède les constantes du jeu (comme les coûts de base ou des distances minimales) en lisant le fichier `config_rules.json`. Ce fichier gère également les fonctions via son nom et une `std::map<std::string, float>` permettant de stocker des bonus ou malus spécifiques pour chaque camp.

Il y a ensuite eu la modélisation des conditions de victoire à l'aide des structures `VictorySet` et `WinConditions`. Une condition de victoire est définie par un type via l'enum `WinType` (ex: nombre de villes, quantité de ressources etc), ce qui permet au moteur de savoir que vérifier. Elle possède également un nom pour la ressource par exemple, un entier pour une quantité ainsi qu'un booléen. Ceci permet de créer de nombreuses conditions de victoires différentes et d'en ajouter facilement. `VictorySet` associe ses conditions à un `WinMode` qui détermine si toutes les conditions sont requises ou une seule parmi la liste. Cela est ajouté dans la configuration globale et permet à l'arbitre à chaque fin de tour de vérifier la victoire d'un joueur.

V. Refactorisation Majeure : Architecture MVC, Standard JSON et Patron Command :

5.1. Migration vers le standard JSON et optimisation du parsing

Lors de la mise en commun initiale de nos codes, des erreurs sont apparues en raison de différences dans nos formats de lecture. Le format txt présentait des problèmes de compréhension et des difficultés de lecture. Durant son stage de l'année précédente chez Orange Business, Naïm a pu manipuler le format JSON et voir sa facilité de structuration des données, notamment pour la lecture en Python. Il a donc proposé de passer vers ce format pour la lecture des données. Pour ce faire, nous avons intégré au sein de notre répertoire [lib/](#) la bibliothèque header-only [nlohmann/json](#) depuis github pour permettre de lire les données json en C++.

```
// Exemple de lecture d'un dictionnaire JSON
nlohmann::json data = nlohmann::json::parse(fichier); // ou fichier >> data
for (const auto& element : data.items()) {
    std::string nomEnnemi = element.key();
    if (data.contains("cout")) {
        auto detailCout = data["cout"];
    }
}
```

Le traitement logiciel de ces données intègre des boucles d'itération par référence constante (`const auto&`), éliminant le coût CPU lié aux copies d'objets JSON. L'utilisation de `data["cle"]` simplifie l'accès aux dictionnaires imbriqués et laisse à la bibliothèque la conversion automatique et sécurisée des types JSON vers les variables C++ correspondantes. L'utilisation des méthodes intégrées comme `.contains()`, `.items()` ou `.value()` permettent de sécuriser les lectures de configurations et de comprendre directement ce qui est lu.

5.2. Refactorisation du Système de Requêtes : Patron Command et `std::variant`

Le moteur de jeu a connu de nombreuses améliorations pendant le développement. Par manque de coordination au départ, nous avons chacun commencé à développer notre propre version du moteur en parallèle, ce qui a mené à deux codes totalement différents. L'une des anciennes versions reposait sur une structure `Action`, couplée à une énumération `TypeAction`. Le problème principal venait du passage des paramètres : cela utilisait un champ fourre-tout `std::string data` qu'il fallait ensuite convertir en entier via `std::stoi` si nous avions finalement besoin d'un entier pour identifier l'index d'une attaque ou d'un soin par exemple. Cette approche avait de gros problèmes de sûreté de typage et n'était pas évolutive.

Le moteur souffrait également d'un problème d'architecture : chaque interaction utilisateur (déplacement, attaque, construction) était pilotée par une fonction dédiée au sein du moteur. L'interface graphique ou la console devaient donc connaître l'intégralité de ces fonctions et leur transmettre les bons arguments. Cela imposait de modifier l'interface du moteur à

chaque ajout de fonctionnalité, et de dupliquer du code lourd et spécifique pour gérer la sauvegarde ou l'envoi réseau de chaque action.

Pour résoudre ce problème, nous avons effectué une refactorisation majeure en implémentant le patron de conception Command avec l'ajout de mécanismes modernes du C++17 : le couple `std::variant` et `std::visit`. L'intégralité des commandes de jeu est désormais encapsulée dans des structures légères et correctement typées contenant leurs propres arguments (ex: `CmdFonderVille`, `CmdAttaque`, `CmdDeplacement`). Ces structures sont réunies au sein d'un type conteneur unique nommé `CommandeJeu` défini par un variant.

L'interface publique du moteur de jeu est ainsi simplifiée à une unique méthode :

```
ResultatAction MoteurDeJeu::soumettreCommande(int pldx, const CommandeJeu& commande);
```

Le retour de la demande est défini par l'énumération `ResultatAction` (qui notifie d'un succès ou d'échecs tel que "Ressources insuffisantes").

Le fonctionnement de cette architecture utilise `std::visit` avec une expression lambda. À l'exécution, `std::visit` regarde le conteneur générique `CommandeJeu`, identifie la structure qu'il possède et redirige vers la surcharge de la fonction `executer` correspondante.

Cette approche offre de nombreux avantages techniques :

- Extensibilité : L'ajout d'une nouvelle mécanique de gameplay ne nécessite plus de modifier les fichiers de transport réseau ou de sauvegarde, car cela manipulent uniquement le type générique `CommandeJeu`.
- Performances : Elle supprime les boucles de conditions ou les structures telles que `switch-case` en laissant le compilateur gérer, garantissant une optimisation maximale des performances.
- Robustesse : Le code est plus sûr, mieux typé et ne tient pas compte de l'interface utilisateur

5.3. Déploiement de l'Architecture MVC : Joueur, Arbitre et Moteur

La conception globale de notre architecture découle directement de notre volonté de maintenir une structure propre, modulaire et évolutive. Au départ du projet, la centralisation excessive de la logique au sein du seul fichier de jeu générerait d'importants conflits d'inclusions croisées et nuisait à la bonne communication interclasse.

Pour assainir cette structure, nous avons introduit la classe de pilotage `MoteurDeJeu` et délimiter les rôles en nous inspirant du patron **Modèle-Vue-Contrôleur (MVC)** : le `MoteurDeJeu` orchestre les flux (Contrôleur), l'`Arbitre` valide les règles de gestion, et la classe `Joueur` incarne le modèle de données. Cet orchestrateur central détient par référence l'ensemble des composants maîtres de la partie (le plateau de tuiles, l'arbitre logique, la liste des joueurs) ainsi que l'indice du tour chronologique en cours, assurant l'indépendance de chaque brique métier.

5.3.1. Le Joueur : Modèle de Données et Conteneur d'État

Cette séparation stricte des responsabilités nous a permis de développer le module **Joueur** de façon autonome et efficace. Le **Joueur** n'a pas à se soucier de la légalité d'une action, ce travail étant réservé à l'Arbitre. Son rôle est d'être un conteneur d'état (inventaire, territoires, unités) et d'appliquer concrètement les conséquences des commandes qui lui parviennent une fois validées.

Gestion des Entités et de l'Économie

Le premier rôle du Joueur est de centraliser la possession des entités sur le plateau. Pour éviter toute duplication mémoire et s'assurer que les unités et les bâtiments manipulés sont bien les mêmes que ceux présents sur la grille, nous les stockons sous forme de listes de pointeurs. Des méthodes dédiées comme **ajouterUnite** ou **perdreUnite** permettent de mettre à jour ce registre dynamiquement au fil des recrutements et des pertes au combat.

Au-delà des unités, la classe **Joueur** encapsule l'économie de sa faction : elle suit l'évolution des ressources globale, la production passive de ses bâtiments et gère dynamiquement la mise à jour de son brouillard de guerre personnel afin de restreindre sa vision aux seules tuiles limitrophes de ses actifs.

Le Joueur comme Exécuteur des Commandes

Une fois qu'une action a été validée par l'Arbitre et transmise par le moteur, c'est la classe **Joueur** qui applique concrètement les modifications sur ses entités. Elle expose pour cela une série de méthodes d'exécution (**Attaquer**, **Soigner**, **Transporter**, **ActiverCamouflage**, etc.) qui font le pont entre l'ordre reçu et la logique interne des entités. Ce processus suit un motif systématique en deux temps :

1. **La délégation logique** : Le Joueur déclenche la mécanique métier appropriée. Pour un appel à **Attaquer**, il délègue la résolution mathématique à la classe statique **Combat::fight**.
2. **La consommation des ressources** : Une fois l'action résolue, le Joueur déduit le coût en Points d'Action (PA) de l'unité applicative via l'appel **unite.setPoint_action(unite.point_action() - 1)**.

La gestion de la hiérarchie de commandement suit la même logique. Les méthodes **RejoindreCommandant** et **QuitterCommandant** relient les pointeurs partagés des unités au **Rank_Commandant** de l'officier, et attribuent inversement le pointeur de l'officier au **Rank_Regulier** de la recrue. Confiner toute cette logique d'exécution dans la classe **Joueur** permet au moteur de rester concis, en lui déléguant les manipulations d'état complexes.

5.3.2. L'Arbitre : Juge Logique et Validation des Règles

Une séparation claire des responsabilités

L'Arbitre joue le rôle de juge suprême de la partie. Le moteur de jeu se contente de router les commandes et d'appliquer les changements d'état, il n'a pas vocation à savoir si une action est légale ou non. Avant chaque exécution, il interroge systématiquement l'Arbitre. Si ce dernier renvoie **false**, l'action est purement et simplement annulée. Nous avons réparti ce travail de vérification en deux domaines distincts : la gestion territoriale

(villes, bâtiments, frontières) d'un côté et la gestion tactique (unités, combats, mouvements) de l'autre.

Sécurité de base et Économie militaire

La première couche de vérification concerne l'appartenance et la solvabilité. Avant toute interaction, la méthode `appartientJoueur` vérifie via un itérateur que l'unité ciblée fait bien partie des troupes du joueur ayant émis la commande. Lorsqu'un joueur tente de recruter une unité, c'est la fonction `peutRecruterUnite` qui entre en jeu. L'Arbitre vérifie si le joueur dispose des fonds nécessaires via la méthode `peutPayer`, qui croise l'inventaire du joueur avec le coût de recrutement configuré. Cette étape garantit en amont l'intégrité de l'économie du jeu.

Les Règles d'Engagement : Attaque, Soin et Décorateurs

Les vérifications les plus complexes concernent la validation des actions offensives (`peutAttaquer`) et médicales (`peutSoigner`). Pour valider un combat, l'Arbitre suit un arbre de décision rigoureux en plusieurs étapes :

- Ressources d'action : Il vérifie d'abord que l'attaquant possède un solde de Points d'Action strictement supérieur à zéro.
- Ciblage : Il confirme que l'attaquant appartient au joueur courant, et que la cible appartient bien à un adversaire.
- Validation des décorateurs : C'est ici que l'architecture par composants montre tout son intérêt. L'Arbitre utilise le transtypage dynamique (`dynamic_cast`) pour détecter si le comportement d'attaque est enveloppé par l'un de nos patrons de conception. Si un `ComportementCooldown` est identifié, il vérifie son état d'activation. S'il détecte un `ComportementConsommable`, il contrôle que les ressources requises sont disponibles dans l'inventaire de l'unité.
- Validation géométrique : Enfin, si toutes les contraintes précédentes sont respectées, l'Arbitre délègue la vérification finale de la portée et du champ de vision au comportement lui-même (`(*it)->PeuxAttaquer(attaque, cible)`).

Pathfinding et Analyse Spatiale

L'un des défis techniques les plus intéressants a été la validation des mouvements et le calcul des déplacements possibles (`getCasesDeplacementPossibles`). Sur un plateau hexagonal avec des coûts de franchissement variables selon le terrain (forêt, montagne, plaine), un simple calcul de distance à vol d'oiseau ne suffit pas.

L'Arbitre encapsule donc une implémentation de l'algorithme de Dijkstra. À partir de la position de l'unité, l'algorithme propage les coûts de mouvement sur les tuiles adjacentes en fonction de la topologie. Les obstacles infranchissables ou les zones sous contrôle ennemi agissent comme des barrières de potentiel, ce qui permet à la fonction de retourner un ensemble exact de cases accessibles pour le budget de PA disponible.

Mécaniques Spéciales et Hiérarchie

L'Arbitre gère aussi la légalité des actions introduites par nos comportements spéciaux :

- Transport et logistique : Pour un embarquement ou un débarquement, l'Arbitre vérifie les points d'action, les temps de recharge et la limite de capacité du véhicule. Pour le débarquement, il s'assure également que la nature géométrique de la tuile de destination est compatible avec le type d'unité (ex: de l'infanterie débarquant sur de la plaine).
- Chaîne de commandement : L'assignation d'une troupe à un commandant exige une proximité stricte. L'Arbitre vérifie que la recrue se trouve sur une case immédiatement adjacente à l'officier, en s'appuyant sur la fonction `Voisins` de notre module d'orientation. Il contrôle ensuite que la limite d'escouade de l'officier n'est pas atteinte avant d'autoriser l'enrôlement.

5.3.3. Le Moteur de Jeu : Orchestration et Cycle de Vie d'une Commande

Le Cycle d'Exécution d'une Commande

Pour toutes les interactions impliquant des unités, le moteur suit un pipeline de validation rigoureux avant de modifier l'état de la partie. Ce pipeline est orchestré en collaboration avec la classe `Arbitre` et s'articule en quatre étapes majeures, exécutées au sein des méthodes `executer()`.

Prenons l'exemple d'une commande offensive (`CmdAttaque`) :

1. Le moteur tente de récupérer l'attaquant et la cible sur le plateau avec `_plateau->getUnite`, puis vérifie via l'arbitre que l'attaquant appartient bien au joueur courant.
2. Ensuite il s'assure que l'unité dispose de suffisamment de points d'action pour initier le combat.
3. Pour cela il extrait la liste des comportements offensifs de l'unité avec `att->Offensive()`, pour isolé l'attaque sélectionnée via l'index fourni dans la commande, puis soumet l'action à l'arbitre via `_arbitre.peutAttaquer`, pour valider les portées, les lignes de vue et les règles spécifiques aux décorateurs.
4. Si toutes les étapes précédentes sont validées, le combat est résolu (`j.Attaquer`). Si les points de vie du défenseur tombent à zéro, le moteur se charge de le retirer proprement du joueur propriétaire ainsi que du plateau de jeu (`_plateau->retirerUnite`).

Diversité des Commandes Tactiques

Grâce à la flexibilité de notre implémentation du type `std::variant`, le moteur est capable de router proprement une grande variété d'ordres tactiques hétérogènes :

- Logistique et mouvement : Le recrutement (`CmdRecrutement`) interagit avec les finances du joueur et les capacités logistiques de la ville avant de placer l'unité sur le plateau. Le déplacement (`CmdDeplacement`) calcule le coût en PA selon la nature de la tuile d'arrivée, déplace l'entité et met à jour sa rotation pour qu'elle « regarde » dans la direction de sa marche.

- Soutien : Les soins (`CmdSoigner`) suivent le même pipeline que les attaques, en puisant dans les `CompSoin` de l'unité. Des commandes dédiées (`CmdRavitailler`, `CmdRavitaillerSurVille`) gèrent le transfert de munitions ou de carburant pour les unités dotées de consommables.
- Comportements spéciaux : Nous gérons aussi l'activation du camouflage (`CmdCamoufler`), la mise en posture défensive (`CmdChangerDefense`) qui augmente le champ de vision et la résistance, ainsi que l'embarquement et le débarquement de troupes (`CmdCharger` / `CmdDecharger`).
- Commandement : Enfin, la hiérarchie militaire est directement manipulable en jeu. Les commandes `CmdEnroller` et `CmdDesenroller` permettent d'attacher ou de détacher dynamiquement une unité régulière à un officier, modifiant ainsi en temps réel la zone d'effet de ses bonus passifs.

Cette architecture par événements, traitée de bout en bout via un visiteur (`std::visit`), nous a permis de découpler totalement la logique interne du jeu de l'interface utilisateur, rendant le moteur bien plus robuste, hautement performant et particulièrement facile à faire évoluer au gré des ajouts de gameplay.

5.4. Conclusion sur le découplage : Une "Vue" interchangeable

L'aboutissement de cette architecture MVC réside dans l'indépendance totale de la Vue. Grâce au système de commandes par `std::variant` et à la centralisation de la logique dans le `MoteurDeJeu`, l'interface n'a qu'un rôle passif :

1. Observation : Elle lit l'état du Modèle (`Plateau`, `Joueur`) pour l'afficher à l'écran.
2. Émission : Elle ne modifie jamais directement les données. Lorsqu'un utilisateur clique sur une unité pour attaquer, la Vue se contente de créer une `CmdAttaque` et de la "pousser" vers la méthode `soumettreCommande` du Contrôleur.

Ce découplage strict nous a permis de développer et de tester toute la logique métier du Wargame avant même d'avoir finalisé l'interface graphique, garantissant ainsi que le moteur de jeu est indépendant de la technologie d'affichage choisie.

VI. Interface :

6.1. L'Interface Graphique (SFML + ImGui) et l'Infrastructure Réseau Multijoueur

6.1.1. Changement technologique : Abandon d'Unreal Engine pour SFML et ImGui

Au départ, le choix était d'utiliser le moteur Unreal Engine pour créer l'interface visuelle du jeu, par le fait d'être très performant et d'avoir un système de script par Blueprints qui évite d'écrire du code C++ pour la création d'interface basique. Cependant, l'intégration de notre code source existant vers Unreal a été très compliqué, car il imposait une structure spécifique avec son propre écosystème logiciel. Pour l'utiliser, nous aurions dû réécrire une grande partie du code.

De plus, pour conserver notre structure actuelle pour github et pour jouer dans le terminal, il aurait fallu dupliquer du code ou créer des liens symboliques complexes sous Windows pour lier les répertoires. Il utilisait également de nombreuses ressources matérielles ce qui ralentissait la compilation et les tests.

Nous sommes donc partis vers un changement radical avec l'utilisation de l'interface de programmation SFML avec la bibliothèque graphique ImGui. Avec SFML, cela permettait d'avoir une couche d'abstraction 2D au-dessus d'OpenGL pour éviter d'écrire du code complexe pour chaque ajout graphique. Ce choix était donc parfait pour construire un jeu en seulement deux mois, car le développement en 3D nécessite beaucoup plus de temps pour comprendre le fonctionnement. ImGui permet lui de générer les menus et les contrôles utilisateurs via un HUD interactif performant.

Pour optimiser le temps de développement, l'interface n'étant pas une priorité pour le projet, nous avons très fréquemment sollicité des modèles d'IA génératives en tant qu'autoformation permettant de comprendre rapidement, de façon générale, les différents concepts utilisés. Cela nous a permis d'accélérer le développement et d'adapter le code à nos besoins.

6.1.2. Architecture de l'affichage

L'architecture graphique est dirigée par la classe `InterfaceManager`, qui correspond au pont entre le backend (moteur, plateau, unité, etc.) et le frontend (le rendu visuel). Elle fonctionne avec une machine d'états `GameState` qui dirige les différentes fenêtres du programme : Menu, Options, Setup, etc. Les configurations graphiques sont synchronisées avec les structures JSON. Pour l'affichage du plateau, les textures issus du répertoire `assets/` sont référencées avec une `std::map` qui lie chaque tuile à son image. La boucle d'exécution `run` écoute en permanence les événements, gère le mouvement de caméra et orchestre les différents appels graphiques.

Le dessin du plateau nécessite un calcul mathématique spécifique : les tuiles possèdent six côtés et non quatre, il faut donc effectuer un décalage géométrique une ligne sur deux (chaque ligne paire ou impaire) pour assembler les hexagones.

Pour cela on utilise les formules suivantes :

```
float posX = W * j + W * 0.5f * (std::abs(i) % 2)
```

→ On commence par regarder si la ligne i est paire ou impaire. Si elle est impaire, on décale toute la ligne vers la droite de la moitié de la largeur de la case ($W * 0.5$) et on la place sur la bonne colonne ($W * j$)

```
float posY = 1.5f * R * i
```

→ Pour pouvoir coller les hexagones, on ne descend pas d'une case entière, mais de $1.5 * R$ (avec R pour le rayon entre le centre et chacun des 6 sommets) pour imbriquer l'hexagone dans le creux des deux hexagones du dessus.

Nous avons également ajouté un système de brouillard de guerre et de découverte de zone. Cette mécanique repose sur l'ajout d'un attribut booléen pour chaque case, permettant au moteur d'enregistrer si la tuile a déjà été explorée par le joueur. Lors de la boucle de rendu, l'interface graphique interroge cet état (*currentState*) pour modifier l'affichage, masquant les zones inexplorées.

Pour sauvegarder l'état global d'une partie (paramètres de carte, inventaires des joueurs, coordonnées précises unités et villes), nous avons créé le module *SaveManager* qui exporte et importe ces données au format JSON afin de maintenir la cohérence de nos formats de fichiers. Cependant, il aurait été intéressant de voir pour sauvegarder le plateau via une feuille de calcul où chaque case posséderait les informations relatives à la tuile du jeu.

6.1.3. Optimisation de l'affichage

Implémentation du View Culling

Après les premiers lancements du jeu sur de grandes cartes, nous avons rencontré de grosses pertes de performances dû à l'envoi d'instructions de dessin pour l'intégralité des tuiles de la carte à chaque frame, même pour celles situées hors de l'écran.

Pour résoudre cela, nous avons implémenté un algorithme de view culling. Le moteur graphique récupère la zone visible à l'écran et lors de la boucle de rendu, le système bloque le dessin de tous les objets situés en dehors de la caméra. Cette optimisation supprime les calculs graphiques inutiles et stabilise les performances.

Ajout de Cache avec dirty flag

En plus de cela, se trouvait un problème de surutilisation du CPU et de calcul inutile correspondant aux données de jeu à afficher. Au départ, la vérification des territoires et des cases achetables était calculée pour chaque tuile visible, pour chaque joueur et à chaque frame. Selon la taille de la carte, cela correspondait à des millions d'opérations par frame et provoquant un lag massif. Pour garantir un minimum de fluidité, l'ajout d'un système de cache (avec la structure *TerritoireCache*) a été mis en place. L'objectif est de ne calculer qu'une seule fois les données complexes et de les stocker en mémoire. Ces données ne sont calculées que si une action les modifie, tel que la fonction d'une ville, l'achat d'une case ou le passage au tour suivant avec l'utilisation d'un drapeau d'invalidation de cache (dirty flag), qui est un booléen décrivant si les données ont été modifiées. L'intérêt est de

passer le coût de lecture de $O(L \cdot C \cdot J)$ (Lignes, Colonnes, Joueurs) à $O(1)$ lors du rendu permettant d'avoir un jeu fluide même en fin de partie.

6.1.4. Déploiement Windows

Le développement a été réalisé sous Windows 11 via l'environnement virtuel WSL pour la compilation. Nous voulions tester les performances de notre moteur sans avoir de bridage dû à la virtualisation ainsi que la possibilité de partager le jeu facilement à n'importe qui sans avoir besoin de Linux.

Nous avons donc configuré la compilation avec MinGW pour traduire le code source C++ vers des exécutables natifs Windows. Cette étape a nécessité de télécharger et d'ajouter manuellement les fichiers de bibliothèques dynamiques .dll de la SFML dans le répertoire de l'application (équivalent des fichiers .so sous Linux qui sont installés nativement).

Pour pouvoir créer facilement un exécutable, nous avons utilisé le logiciel open-source Inno Setup en rédigeant un script au format .iss afin de créer un assistant d'installation .exe.

6.1.5. Infrastructure réseau multijoueur synchrone

Pendant le développement de l'interface graphique est venu l'idée d'ajouter des fonctionnalités multijoueurs, notamment grâce à la simplicité de la SFML qui permet de gérer le réseau. Cela nous a permis de se documenter sur les mécanismes liés au réseau et à son fonctionnement. Un problème de désynchronisation du jeu apparaissait très régulièrement pendant la partie lorsque l'interface graphique n'envoyait pas de paquets au bon moment.

Pour cela, nous avons utilisé le composant `sf::Packet`, en encapsulant les données sur des entiers de taille fixe à 32 bits (`sf::Int32`). Cela permet de partager plusieurs types différents en une unique transaction réseau. Chaque paquet possède une étiquette qui identifie le paquet depuis l'énumération `PacketType`, permettant de comprendre les données reçues.

Cela utilise donc les classes de SFML :

- `sf::TcpSocket` et `sf::TcpListener` pour écouter et créer un lien permettant l'échange de données.
- `startHosting` et `connectToHost` pour créer une liaison entre l'hôte de la partie (qui est client et serveur) et les clients distants.
- La méthode `update()`, appelée à chaque frame par l'interface graphique pour examiner l'état du réseau, valider les connexions ainsi qu'appeler les méthodes `receiveData` et `sendData` pour synchroniser la partie après chaque action entre tous les joueurs.

Pendant le menu multijoueur, l'interface désactive le contrôle de la configuration de la partie pour les clients et laisse uniquement l'hôte modifier les paramètres. Dès que l'hôte applique une modification du jeu ou qu'un joueur effectue une action, le paquet réseau correspondant est envoyé à la fin de la fonction pour pouvoir synchroniser les joueurs.

Cependant, n'étant pas un professionnel du réseau, il se peut qu'il y ait de nombreuses failles de sécurité et la possibilité d'injecter des données. Notre code aurait donc

probablement besoin de relecture et de modification pour éviter cela avant utilisation en dehors d'un contexte de développement.

6.2 Interface console

6.2.1. L'aboutissement du modèle MVC

L'architecture de notre projet ayant été strictement pensée autour du patron de conception MVC (Modèle-Vue-Contrôleur), le cœur de notre jeu (le Moteur, l'Arbitre, les Joueurs) est totalement indépendant de l'affichage. Le fichier `main_console.cc` constitue notre implémentation concrète de la "Vue" et du "Contrôleur" sous la forme d'une interface en ligne de commande.

6.2.2. Initialisation Data-Driven

Au lancement de l'exécutable, le programme instancie le `MoteurDeJeu` et lui injecte les chemins vers l'ensemble des fichiers de configuration JSON (règles, ressources, bâtiments, villes, unités, tuiles) via la structure `GameConfigFiles`. L'interface gère ensuite le paramétrage de la partie en interagissant avec l'utilisateur : choix du nombre de joueurs, sélection des factions dynamiquement récupérées depuis le moteur (`moteur.getFactionsAvailable()`), choix de la graine de génération (`seed`) pour l'aléatoire, et sélection des conditions de victoire. Une fois ces données récoltées, la méthode `moteur.initGame()` est appelée pour générer le plateau et placer les capitales.

6.2.3. La Boucle Principale de Jeu

Le cœur de l'expérience utilisateur réside dans la boucle `while (!moteur.isPartieTerminee())`. À chaque itération, l'interface affiche un résumé complet de l'état du joueur actif via `afficherEtatJoueur` : ses ressources, la position et les points d'action de ses unités, ainsi que l'état de ses villes.

L'utilisateur navigue ensuite via un menu interactif. Lorsqu'il choisit de réaliser une action tactique (option 2), le flux est redirigé vers la fonction `menuAction`. Cette fonction agit comme un véritable Contrôleur :

1. **Récupération de l'intention** : Elle demande à l'utilisateur quelle action il souhaite effectuer.
2. **Assistance visuelle** : Pour des actions complexes comme la rotation ou l'attaque, le contrôleur interroge le moteur pour afficher des informations d'aide. Par exemple, il calcule les cibles valides pour une attaque ou affiche le cône de vision interactif de l'unité sélectionnée (`afficherConeVision`).
3. **Génération de la commande** : Les entrées de l'utilisateur (coordonnées, index de compétence) sont encapsulées dans les structures de notre `std::variant` (ex: `CmdAttaque`, `CmdDeplacement`).
4. **Soumission au moteur** : La commande est envoyée au moteur via `moteur.soumettreCommande(pldx, cmd)`, et le retour (`ResultatAction`) est traduit en un message clair pour l'utilisateur via `afficherResultat` (ex: "ÉCHEC Ressources insuffisantes").

VII. Implémentation :

Dans l'optique de coller à la réalité du développement d'un studio de jeu vidéo, nous avons élaboré un univers complexe, doté d'un *lore* (histoire de fond) et d'une direction artistique (DA) claire. Ce travail de contextualisation a servi de fil conducteur pour éprouver la robustesse de notre architecture.

7.1. La construction de l'univers (*Worldbuilding*)

Nous avons écrit une mise en contexte narrative spécifique pour l'univers spatial que nous souhaitions déployer. Le jeu prend place dans la galaxie fictive de *Lerne*, où trois factions s'affrontent pour le contrôle de la zone. Loin d'être purement cosmétique, cette scénarisation a permis de mettre à l'épreuve notre moteur avec la gestion de trois factions asymétriques ne possédant pas les mêmes unités. Cela nous a poussés à concevoir des fichiers de configuration JSON complexes pour modéliser ces spécificités.

7.2. Direction Artistique et Asset Pipeline

Au vu du temps disponible pour la création de ce projet, et de nos compétences graphiques initiales, nous sommes partis à la recherche d'outils capables de garantir une cohérence visuelle globale.

Pour la représentation de nos tuiles (planètes et objets spatiaux), nous avons exploité le travail de *Deep-Fold* (sur la plateforme itch.io), créateur d'un générateur de planètes en open source qui nous a grandement servis (*PixelPlanets*). En partant de cet outil, nous avons pu générer l'intégralité des décors dont nous avons besoin.

Pour les bannières et les unités des différentes factions, nous avons utilisé le générateur d'images de l'IA *Gemini*. Ce processus nous a confronté à la complexité de l'ingénierie des requêtes ou le prompt engineering, afin de maintenir une charte graphique uniforme au fil des générations. Après de nombreux tests, nous avons stabilisé une structure de prompt rigoureuse qui a assuré la continuité de notre DA (cf. annexe E) :

- **"2D vector art, flat design, cel-shading icon of..."** : Définit le style graphique.
- **"Direct nose-on frontal view / 3/4 front view"** : Verrouille la perspective pour que l'objet fasse face au joueur au lieu de fuir vers le fond de l'écran.
- **"The rear thrusters and engines are completely hidden from view"** : Contrainte de sécurité essentielle empêchant l'IA de dessiner des réacteurs arrière, ce qui aurait retourné le vaisseau par rapport au sens de lecture.
- **"Thick dark outlines, sharp edges, no complex gradients"** : Assure des contours noirs épais et des aplats de couleurs nets, indispensables pour que le sprite ressorte clairement au-dessus du fond des planètes.
- **"Gaming map UI sprite style, isolated on a solid light gray background"** : Indique qu'il s'agit d'un *asset* de jeu épuré, isolé sur fond uni pour faciliter le détourage au format PNG transparent.

7.3. Équilibrage des données (*Game Design*)

Enfin, l'équilibrage des statistiques constitue un pilier majeur de la viabilité d'un *wargame*. N'ayant pas l'expertise de Game Designers, nous avons collaboré avec l'IA Claude pour calibrer l'économie globale et les caractéristiques de nos unités.

L'IA nous a généré une matrice de données (fichier .xlsx) contenant l'intégralité de l'équilibrage(cf. annexe B.2). Après une phase d'audit, de vérification et de correction des incohérences, nous avons transposé ces valeurs sous forme de fichiers de configuration JSON (cf. annexe B.1). Cette approche Data-driven rend notre moteur totalement modulaire : modifier la puissance d'une unité ou le coût d'un bâtiment se fait instantanément dans le fichier texte, sans nécessiter aucune recompilation du code C++.

VIII.Test :

8.1. La nécessité d'une validation rigoureuse

Développer un wargame universel implique une grande complexité algorithmique : calculs de vision hexagonale, application de malus de moral, décorateurs de temps de recharge, etc. Pour garantir la stabilité de notre architecture et éviter les régressions lors de l'ajout de nouvelles fonctionnalités, nous avons mis en place une suite de tests unitaires centralisée dans le fichier `main_test.cc`.

Pour ce faire, nous avons utilisé la bibliothèque de tests légère et moderne `doctest.h`.

8.2. Isolation et *Mocking* des données

L'un des défis des tests unitaires est d'isoler le composant à tester sans avoir à charger l'ensemble de l'environnement de jeu (fichiers JSON, plateau complet, interface). Pour pallier cela, nous avons développé une fonction utilitaire `createTestUnite`. Cette fonction agit comme un mock: elle génère à la volée des pointeurs intelligents vers des unités "vides" aux statistiques paramétrables, ce qui nous permet de tester les interactions de combat ou de mouvement de manière purement mathématique et isolée.

8.3. Couverture des Tests

Notre suite de tests couvre l'exhaustivité des mécaniques critiques du moteur :

- **Mouvement et Algorithmique spatiale** : Nous vérifions que les classes `CompMouv` valident correctement les cases à portée, et que la fonction complexe `ConeVision` génère bien la bonne taille de matrice sans déborder derrière l'unité.
- **Comportements et Décorateurs** : Les templates `AvecCooldown` et `AvecConsommable` sont testés.
- **Mécaniques de Combat et Moral** : Nous simulons des affrontements de bout en bout. Nous testons l'orientation. Le système de moral est également validé aux valeurs extrêmes.
- **Hiérarchie** : Un test dédié s'assure que l'enrôlement sous un `Rank_Commandant` transfère correctement les bonus passifs vers les statistiques temporaires de la troupe régulière lors d'un calcul de combat.

Cette démarche nous a permis de vérifier la validité de notre code pour des fonctions parfois très compliquées à pouvoir tester rapidement en jeu.

IX. Intelligence Artificielle par Apprentissage Profond par Renforcement (Deep RL)

9.1. Objectifs de l'expérimentation et framework technologique

Après avoir terminé l'infrastructure du jeu, notre intérêt pour les systèmes intelligents nous a poussé à mener une recherche sur l'intégration d'une Intelligence Artificielle. L'objectif était d'implémenter un agent de Deep Reinforcement Learning (apprentissage profond par renforcement) pouvant apprendre par lui-même à jouer à un jeu complexe avec de nombreuses mécaniques telles que le nôtre.

Pour travailler sur cela, nous avons regardé de nombreux documents, conversé avec des modèles d'IA génératives ainsi qu'utiliser nos connaissances suite au cours d'Introduction au Machine Learning suivi cette année.

Nous avons utilisé ici l'algorithme PPO (Proximal Policy Optimization) fourni par la bibliothèque stable-baseline3 (library d'algorithmes d'apprentissage par renforcement). C'est notamment cet algorithme qui a été utilisé par OpenAI pour battre les champions du monde du jeu Dota 2. La majorité des frameworks et library d'IA utilise aujourd'hui Python, nous avons donc orienté le développement pour continuer sur les notions vues en cours en utilisant PyTorch.

9.2. Architecture de liaison (Bindings C++ / Python)

L'objectif de cette implémentation est de conserver les performances du C++ pour faire tourner le moteur du jeu en entraînant l'IA via Python.

Voici l'architecture mise en place :

1. **Moteur du jeu C++** : La logique du jeu pur, exécuté sans l'interface graphique donc sans SFML
2. **Fichier `bindings.cc`** : Cela utilise la bibliothèque Pybind11 qui permet de traduire du code C++ en Python en créant des liens. Pybind crée un pont entre les deux langages et fournit une bibliothèque binaire nommée `wargame_env.so` qui contient les différentes fonctions du jeu pour Python. Elle traduit le jeu en différentes commandes exécutables par l'interpréteur Python.
3. **Fichier `wargame_gym.py`** : C'est ici qu'est géré l'IA en implémentant l'architecture Gymnasium, qui permet d'avoir une interface universelle en utilisant les mêmes fonctions de base : `reset()` et `step(action)`, pour le Deep Learning. Ce script possède le moteur de jeu dans une classe et configure une perception visuelle avec une caméra locale et une minimap globale compressée via OpenCV. Cela permet à l'IA d'avoir une stratégie globale et de pouvoir jouer un plateau de n'importe quelle taille. C'est également dans ce fichier que l'on gère le système de récompense pour orienter le comportement de l'agent.
4. **Fichier `CMakeLists.txt`** : Ce script compile le moteur et permet de faire tourner et de gérer plusieurs instances du jeu en même temps sans saturer la mémoire RAM et sans perte de performance avec un rendu visuel.

5. **Fichier `train_ai.py`** : De la même façon que ce qui a été vu dans le cours d'introduction au machine learning, ce script organise l'apprentissage et gère le modèle d'IA.

9.3. Difficultés techniques rencontrées et optimisations systèmes

Contrainte de performance (CPU vs GPU)

L'optimisation des ressources disponibles est primordiale pour entraîner l'IA le plus efficacement possible. Après plusieurs tests avec différents matériels, Naïm a pu remarquer qu'utiliser sa carte graphique (Nvidia RTX 2070 Super) donnaient des performances inférieures d'un entraînement avec son processeur uniquement (Ryzen 5 3600 ou Ryzen 5 7530U).

Cela est donc contraire avec ce qui a été vu en cours, mais cela s'explique par une différence majeure : en Deep Learning classique, les données d'entraînement existent déjà sur le disque dur et on envoie directement une grande quantité d'informations à la carte graphique. Dans notre cas, les données n'existent pas encore et le jeu doit être simulé en temps réel par le moteur C++ et tourne donc sur le CPU pour effectuer les calculs logiques.

Le temps de transfert pour envoyer les données générées de la RAM vers la VRAM via le bus PCIe est plus long que le temps nécessaire au processeur pour faire le calcul lui-même. La carte graphique effectuait les calculs de manière quasi instantanée et passait son temps à attendre le processeur. Pour optimiser cela, le mieux est donc de laisser les calculs directement sur le CPU.

Parallélisation via Multiprocessing

En regardant l'utilisation du CPU, on a pu remarquer que le processeur n'était pas entièrement utilisé car l'environnement n'utilisait qu'un seul processus. Nous avons donc activé le multiprocessing pour faire tourner plusieurs processus en parallèle, chacun ayant sa propre instance du jeu. Cette parallélisation a permis d'optimiser l'usage CPU et d'atteindre une vitesse de 120 FPS contre 30 FPS pour le GPU (les FPS correspondent aux nombres de steps, c'est-à-dire les actions calculées et simulées par le jeu par seconde).

Coupe-circuit

Pendant l'entraînement, un problème est apparu, l'IA avait tendance à effectuer des boucles d'actions impossibles (comme recruter une unité sans les ressources) en les enchaînant sans passer son tour, ce qui créait un circuit infini et bloquait le programme. Pour que l'entraînement avancé, l'ajout d'un coupe-circuit pour éviter que l'IA n'enchaîne des actions impossibles a été nécessaire. Si on détecte une répétition d'erreurs, le coupe-circuit force le passage au tour suivant.

Equilibrage des récompenses

La gestion des récompenses est également une étape nouvelle et cruciale qui nécessite de nombreux tests et ajustements pour entraîner l'IA de différentes manières et trouver le bon compromis. Pendant plusieurs sessions d'entraînement, l'IA resta indéfiniment, après plusieurs millions de steps, à un score négatif.

Par rapport à la durée d'une partie : le wargame dure au maximum 150 tours (valeur définie dans le fichier de configuration) et une IA peut effectuer jusqu'à 20 actions avant de passer son tour. Donc si une partie dure en moyenne 100 tours avec 10 actions par tour, cela représente 1000 étapes (steps) par partie jouée.

Pour atteindre un objectif de deux millions de steps par exemple, l'IA doit simuler 2000 parties. Avec le rythme de 125 FPS, il faut environ 4 heures et 26 minutes d'exécution ($2 \text{ millions de steps} / 125 \text{ FPS} = 16\,000 \text{ secondes}$). Pour le moindre changement dans les récompenses ou autre, cela nécessite de nombreuses heures d'entraînement à nouveau. Pourtant cela reste plutôt faible pour que le réseau de neurones comprennent toutes les subtilités du jeu, sachant que la carte est générée de façon procédurale.

9.4. Démarche d'apprentissage et bilan pédagogique

L'objectif de cette recherche était d'apprendre et de comprendre de nouvelles structures dans le développement d'IA complexes. Pour faire cela, l'utilisation de l'IA générative comme professeur pour l'autoformation et de vidéos youtube a été très importante. Le développement se faisant étape par étape, pour pouvoir être confronté aux difficultés et comprendre pourquoi telle modification de code était à effectuer. Cela a permis d'assimiler des concepts de l'apprentissage profond par renforcement et de tester le système de récompense.

Conclusion :

Bilan technique

Ce stage effectué dans le cadre de notre Licence 3 d'Informatique à Angers sous la direction de M. Igor STÉPHAN, nous a permis de concevoir de toutes pièces un moteur complet d'universal wargame. Cela nous a permis de nous perfectionner en C++ et de découvrir de nouveaux concepts d'architecture logicielle.

Nous avons ici construit notre logiciel de façon à être piloté par les données (Data-Driven). Grâce à l'utilisation du patron de conception Fabrique (Factory) et l'ajout de comportements aux unités, notre jeu est devenu entièrement configurable via des fichiers JSON. Le moteur est donc devenu générique, permettant d'avoir plusieurs univers sans besoin de recompiler le code.

Notre projet utilise notamment l'architecture Modèle-Vue-Contrôleur (MVC) permettant l'amélioration du jeu et l'ajout de fonctionnalités très simplement. En a découlé une refactorisation en utilisant les fonctionnalités modernes du C++ avec l'implémentation du patron de conception Command grâce à `std::variant`. Cela permet d'avoir un code performant et compréhensible qui, avec cela, a permis l'ajout de l'interface graphique via SFML et ImGui facilement.

Bilan organisationnel et méthodologique

Cela a permis de comprendre les difficultés de travailler à plusieurs sur le même projet, l'importance de bien se répartir les tâches ainsi que l'obligation d'avoir une architecture de départ claire pour éviter les problèmes de compréhension et pouvoir développer dans la même direction.

Nous avons également pu produire un diagramme de Gantt très différent des prédictions que nous avons proposées en introduction (cf. annexe C).

Perspectives et ouvertures

Bien que le temps imparti nous ait imposé de nous concentrer sur les fonctionnalités essentielles, nous avons pu écrire les bases architecturales qui permettent un code optimisé et évolutif. Grâce à notre code, il est très simple d'ajouter des mécaniques ou des fonctionnalités sans avoir besoin de modifier de nombreux fichiers.

L'ajout d'une Intelligence Artificielle d'Apprentissage Profond par Renforcement est une perspective intéressante, car nous n'avons ici qu'une ébauche de développement et cela demanderait plus de temps, notamment pour l'entraînement.

De nombreux ajouts de fonctionnalités sont envisageables, que ce soit pour l'interface graphique ou la création de nouveaux comportements d'unités (attaques de zone, mécaniques de trahison etc.). Avec le code actuel, l'ajout de telles fonctionnalités sera facile à implémenter, sans avoir besoin de modifier toute l'architecture du système.

Il est également très simple de créer de nouveaux univers en plus de celui dans l'espace et des pirates. Cela demandera des compétences particulières en game design pour équilibrer le jeu et le rendre intéressant à jouer. C'est là tout l'intérêt de cet universal wargame : la facilité de créer des instances dans différents univers, avec un moteur performant et stable.

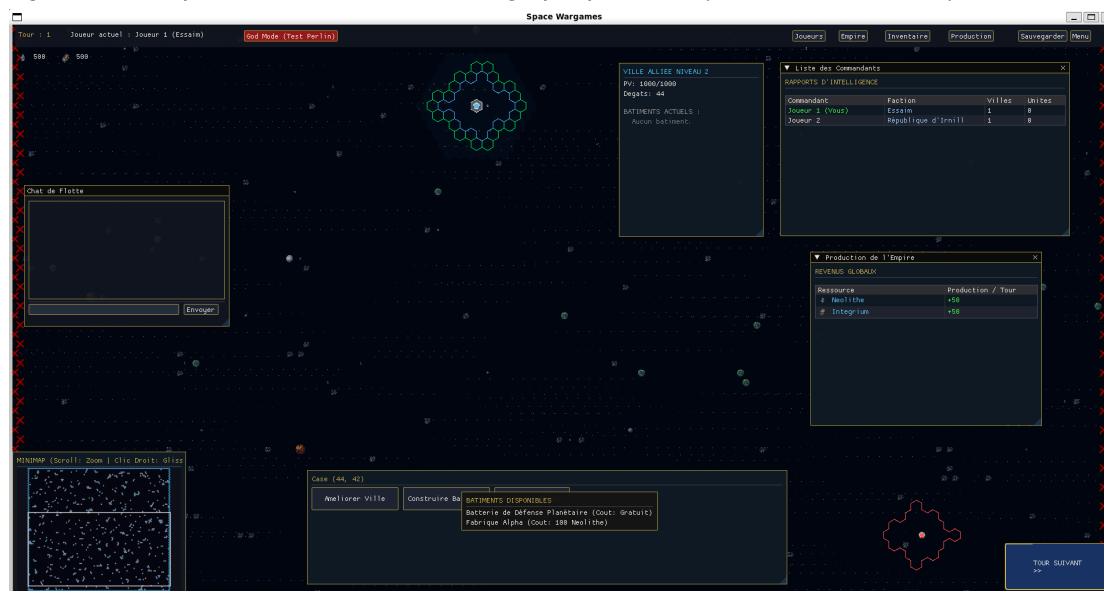
Webographie : Bibliothèques Logicielles Tierces utilisées

- **Lohmann, N. (JSON for Modern C++)**
 - Ressource : <https://github.com/nlohmann/json>
 - Description : Bibliothèque header-only intégrée au répertoire lib/. Utilisé pour la lecture de nos fichiers JSON en C++ pour suivre le design Data-Driven du projet.
- **Gomila, L. (Simple and Fast Multimedia Library)**
 - Ressource : <https://github.com/SFML/SFML>
 - Description : Framework open-source utilisé pour l'affichage de la fenêtre, de contenu graphique en 2D et des entrées utilisateurs.
- **Cornut, O. (Dear ImGui)**
 - Ressource : <https://github.com/ocornut/imgui>
 - Description : Bibliothèque d'interface utilisateur graphique simple utilisée avec SFML. Permettre la création des menus.
- **Peck, J. (FastNoiseLite)**
 - Ressource : <https://github.com/Auburn/FastNoiseLite>
 - Description : Générateur de bruit header-only. Elle est utilisé pour implémenter l'algorithme de Perlin dans la classe WorldGenerator pour créer des plateaux de façons procédurale
- **Kirilov, V. (doctest)**
 - Ressource : <https://github.com/doctest/doctest>
 - Description : Framework de tests unitaires léger et moderne utilisé dans main_test.cc pour valider les algorithmes des unités.
- **Russel, J. (Inno Setup)**
 - Ressource : <https://github.com/jrsoftware/issrc>
 - Description : Logiciel permettant de créer avec un script .iss un assistant d'installation .exe.

Annexes :

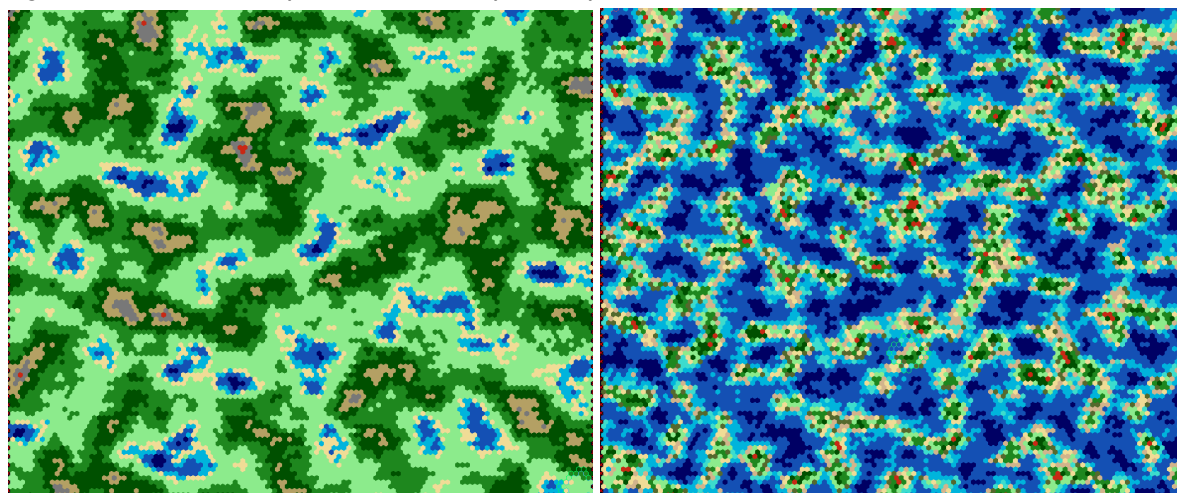
Annexe A : Interfaces et rendus visuels du moteur de jeu

Figure A.1 : Capture d'écran de l'interface graphique finale (SFML + Dear ImGui)



Cette capture d'écran présente le rendu visuel du jeu avec l'utilisation de SFML et Dear ImGui. On peut y voir la grille hexagonale ainsi que le menu avec les différentes fonctionnalités.

Figure A.2 : Génération procédurale d'un plateau par bruit de Perlin



Cela permet de voir la carte générée de manière procédurale par la classe WorldFactory en utilisant l'algorithme de bruit de Perlin fourni par la library FastNoiseLite. Cela lit le fichier JSON pour répartir les tuiles de façon cohérente avec plusieurs presets. Chaque couleur est associée à une tuile, ici pour le jeu de pirate, on peut voir la différence entre un preset qui crée de grands continents avec des lacs ou un monde entièrement d'eau avec plein d'île.

Annexe B : Implémentations techniques et configurations

Figure B.1 : Fichier JSON de configuration de l'unité "Chasseur Amélioré"

```
{
  "nom": "Chasseur Amélioré",
  "hp": 100,
  "action": 0,
  "poids": "",
  "symbole": "",
  "cout": {
    "Neolithe": 450,
    "Ilektraerium": 500
  },
  "cout_entretien": {},
  "vision": 0,
  "fov": 0,
  "comportements": [
    { "type": "MouvementTerrestre", "mouvement_par_tour": 5 },
    { "type": "MouvementVolant", "mouvement_par_tour": 5 },
    { "type": "AttaqueMelee", "degats": 35 },
    { "type": "AttaqueDistance", "degats": 25, "portee": 3, "portee_mini": 2 }
  ],
  "texture": "assets/Space/Unite/Chasseur_Ameliorer.png"
},
```

Cet extrait de code permet de comprendre comment fonctionne notre conception pilotée par les données (Data-Driven Design). Ce fichier JSON externe est lu au démarrage pour spécifier les unités et associer les bons comportements.

Figure B.2 : Matrice d'équilibrage jeu dans l'espace

X Unités Régulières (communes aux 3 factions)						
Statistique / Comportement	Chasseur	Chasseur Amélioré	Croiseur	Frégate	Transporteur Tactique	Vaisseau de Soutien
--- COÛT D'INVOCATION ---						
Néolithe	400	450	545	355	400	450
Integrium	—	—	—	425	320	—
Ilektraerium	535	500	435	280	—	—
Vitriolite	—	—	—	—	—	—
Héliosium	—	—	—	—	255	400
--- STATISTIQUES ---						
HP	100	100	100	150	125	125
Mouvement	3	5	4	4	4	5
Dégâts Mêlée	35	35	—	—	—	—
Dégâts Distance	—	25	25	25	—	—
Dégâts Indirect	—	—	—	—	—	—
Tours Infection	—	—	—	—	—	—
Armure (%)	—	—	—	20	—	—
Boucliers	—	—	—	—	2	2
Soin Direct	—	—	—	—	—	30
Soin Indirect	—	—	—	—	—	—
Tours Regen	—	—	—	—	—	—
Portée	1	3	3	3	—	2
Portée Min	—	2	2	2	—	—
Transport Max	—	—	—	—	3	—
Furtivité (tours)	—	—	—	—	—	—
--- COMPORTEMENTS ---						
Mouvement Terrestre	✓	✓	✓	✓	✓	✓
Mouvement Volant	—	✓	—	—	—	✓
Mouvement Marin	—	—	✓	✓	✓	—
Attaque Mêlée	✓	✓	—	—	—	—
Attaque Distance	—	✓	✓	✓	—	—
Attaque Indirecte	—	—	—	—	—	—
Défense Armure	—	—	—	—	—	—
Défense Bouclier	—	—	—	—	✓	✓
Soin Direct	—	—	—	—	—	✓
Soin Indirect	—	—	—	—	—	—
Transport	—	—	—	—	✓	—
Furtivité	—	—	—	—	—	—

Voici un extrait de la feuille de calcul élaborée avec l'IA Claude pour équilibrer l'économie globale et les caractéristiques des unités. Ces valeurs ont ensuite été transformées vers le fichier de configuration JSON.

Figure B.3 : Patron Command avec utilisation de `std::variant`

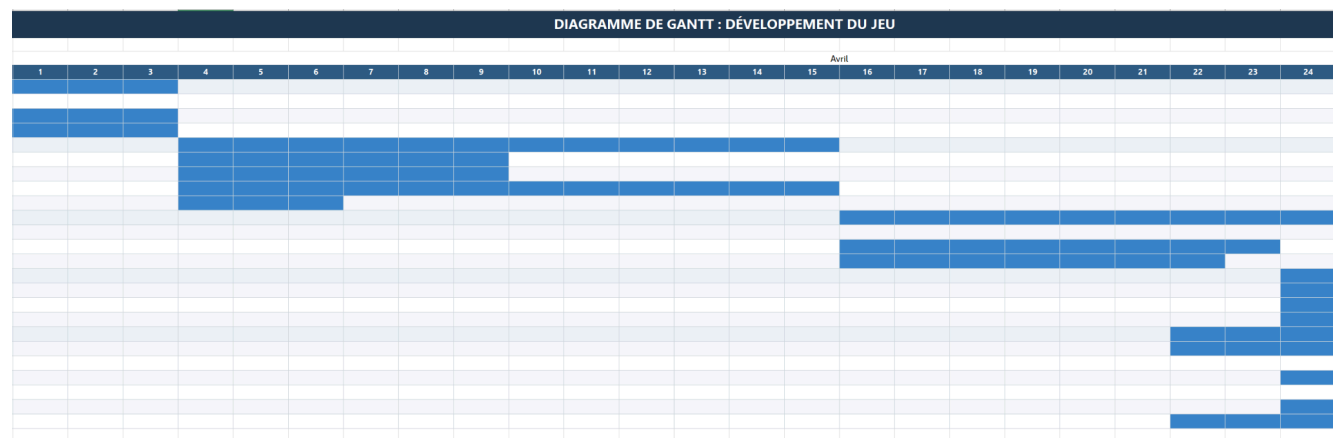
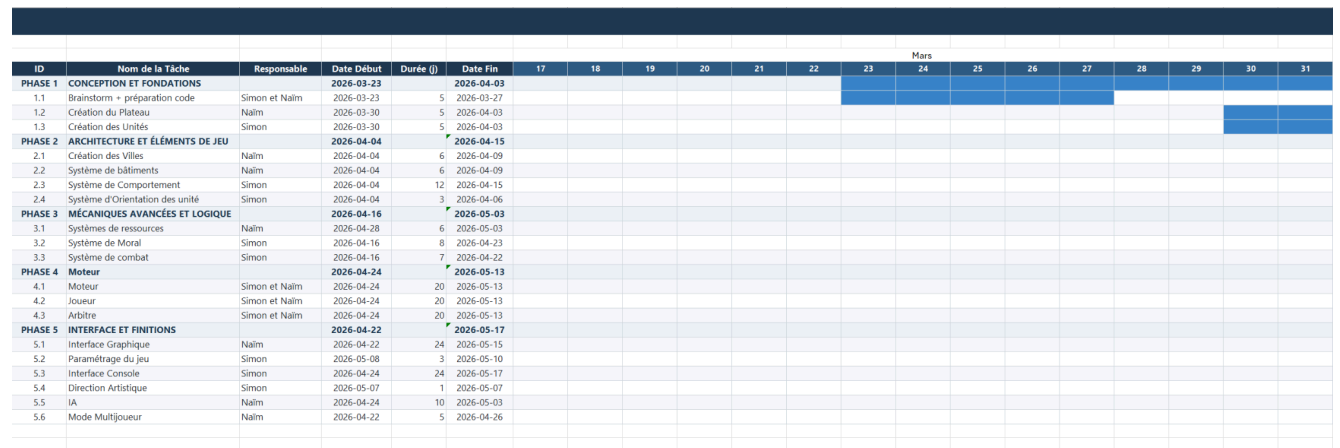
```
struct CmdFonderVille { int x, y; std::string nomVille; };
struct CmdAcheterCase { int x, y; };
struct CmdAmeliorer { int x, y; };
struct CmdDeplacement { int xSrc, ySrc, xDest, yDest; };
struct CmdAttaque { int xSrc, ySrc, xDest, yDest, indexAtt; };
struct CmdRotation { int x, y; direction dir; };
struct CmdRecrutement { int x, y; std::string nomUnite; };
struct CmdConstruction { int x, y; std::string nomBatiment; };
struct CmdSoigner { int xSrc, ySrc, xDest, yDest, indexSoin; };
struct CmdCamoufler { int x, y; };
struct CmdCharger { int xPassager, yPassager, xTransport, yTransport; };
struct CmdEnroler { int xCommandant, yCommandant, xRecrue, yRecrue; };
struct CmdChangerDefense { int x, y; };
struct CmdRavitaillerSurVille { int x, y; };
struct CmdRavitailler { int xSrc, ySrc, xDest, yDest; };
struct CmdDetruireUnite { int x, y; };
struct CmdFinTour {};

// variant : contient une seule des structures suivantes
using CommandeJeu = std::variant<
    CmdFonderVille,
    CmdAcheterCase,
    CmdAmeliorer,
    CmdDeplacement,
    CmdAttaque,
    CmdRotation,
    CmdRecrutement,
    CmdConstruction,
    CmdSoigner,
    CmdCamoufler,
    CmdCharger,
    CmdEnroler,
    CmdChangerDefense,
    CmdRavitaillerSurVille,
    CmdRavitailler,
    CmdDetruireUnite,
    CmdFinTour
>;

ResultatAction soumettreCommande(int pldx, const CommandeJeu& commande);
```

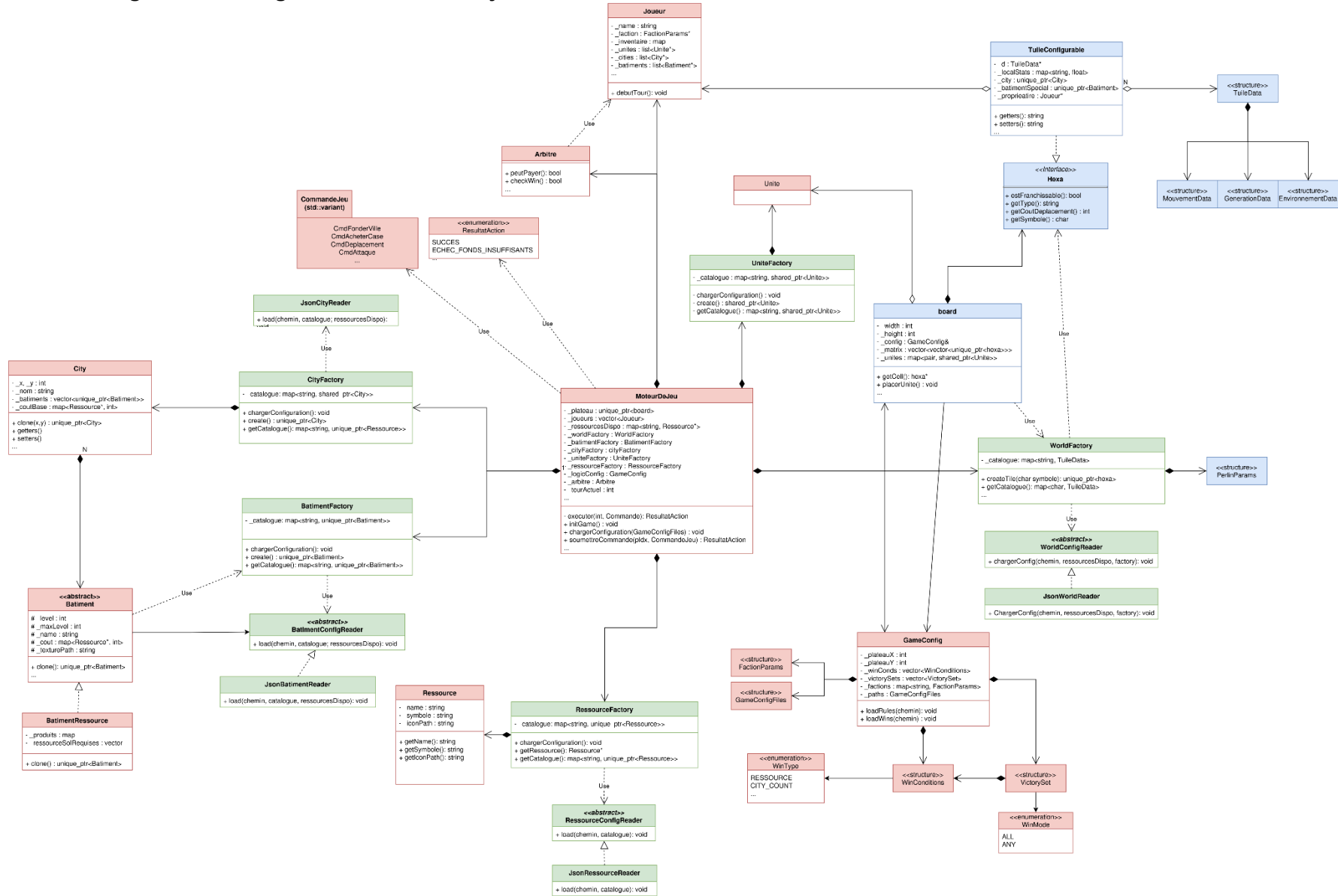
Ce bloc de code permet de voir l'implémentation du patron de conception Command avec l'utilisation de `std::variant`. Toutes les interactions possibles de l'utilisateur sont décrites dans le conteneur unique `CommandeJeu` et permettent d'utiliser comme interface la méthode `soumettreCommande` située dans le moteur.

Annexe C : Diagramme de Gantt



Le fait de retravailler notre diagramme de Gantt a mis en lumière un manque d'anticipation concernant certaines fonctionnalités, ainsi qu'une sous-estimation de la charge de travail pour d'autres. De plus, la transversalité de certains composants, comme l'interface graphique ou le moteur de jeu, a nécessité des modifications continues tout au long du développement.

Annexe D : Diagramme UML global du moteur de jeu



Le diagramme ci-dessus représente la modélisation UML globale des classes contenues dans le dossier jeu/ de l'application. Elle est structurée autour de la classe MoteurDeJeu qui orchestre tous les différents modules.

Dans l'objectif de garder une certaine lisibilité, certains liens n'ont pas été ajoutés au rendu visuel. Prenons l'exemple avec la classe Ressource qui est partagée avec une grande quantité de composants du projet. Nous pouvons cependant les retrouver dans le typage des attributs et des méthodes au sein des différentes classes.

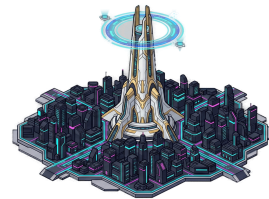
Annexe E : Ressources visuelles - Space Wargame



Chasseur Améliorer



Neolithe



Capital

Exemples de rendus générés par intelligence artificielle selon notre modèle de prompt.

Annexe F : Gestion des versions

L'intégralité du code source de notre projet d'universal wargame, avec l'historique des commits, les branches de développement, les fichiers de configuration JSON ainsi que les assets graphiques pour le jeu dans l'espace est accessible sur notre dépôt GitHub.

Github : <https://github.com/Nems0s/universal-wargames>